Minimax algorithm II



Zola Mahlaza

Department of Computer Science

Feb 2026

Topics

- The Minimax Algorithm
  - Minimax completes depth-first exploration of game tree

Introduction

- If opponent does not play first:

Rely on utility function for ‘leaf’ states

```
function MINIMAX-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move ← v2, a
    return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move ← v2, a
    return v, move
```

- If opponent does not play first:

min/max for each subtree

```
function MINIMAX-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move ← v2, a
    return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move ← v2, a
    return v, move
```

Minimax algorithm

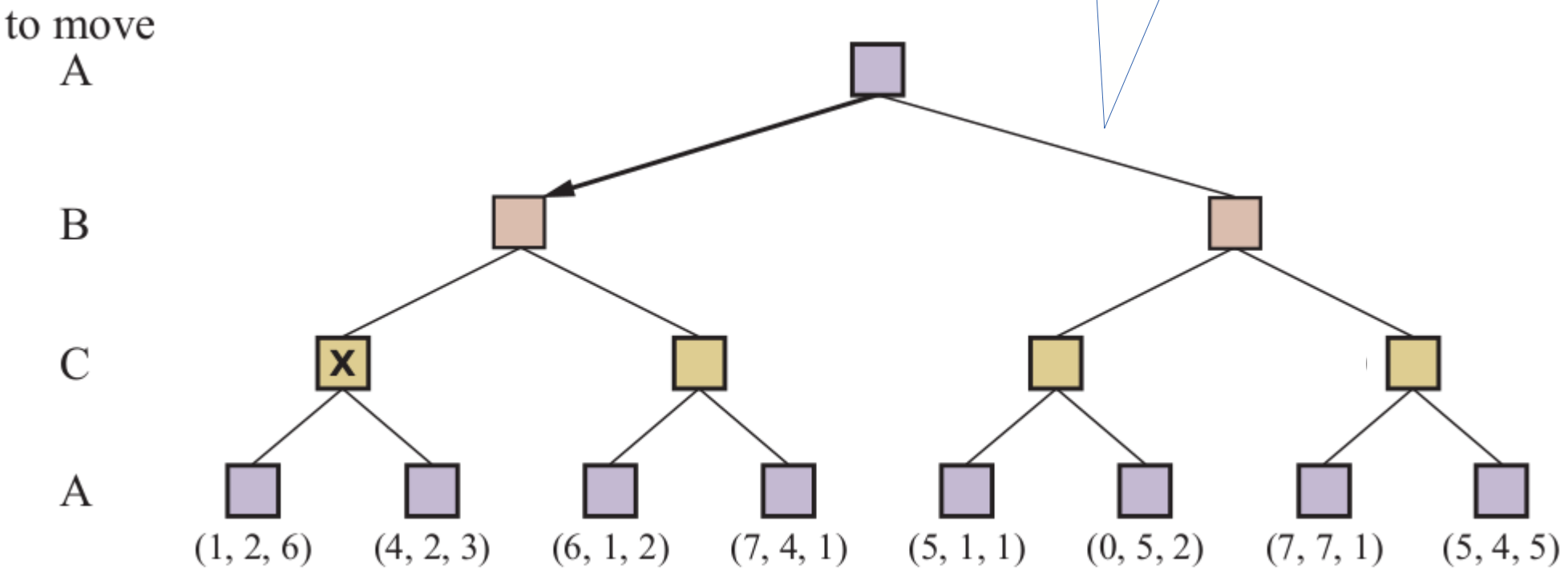
- Time complexity is $O(b^m)$ where b = branching factor, m = max depth
- Space complexity is O(bm).
  - Once the minimax value is computed for upper level, dispose of the nodes

Multiplayer games

- A number of games are multiplayer
- Extending minimax:
  - single value in node becomes a vector
- In leaf nodes, utility of the state is given from each player's viewpoint
  - UTILITY function can return a vector

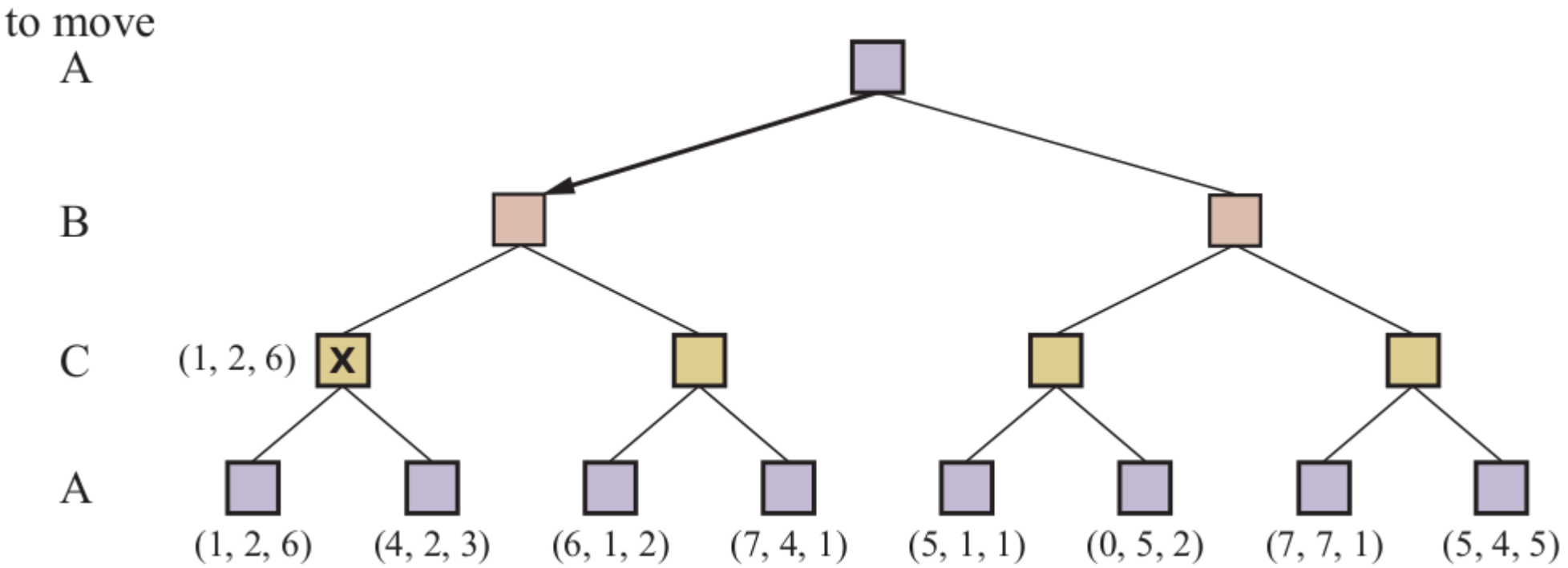
Multiplayer games

How to calculate value of Non-terminal states?



Multiplayer games

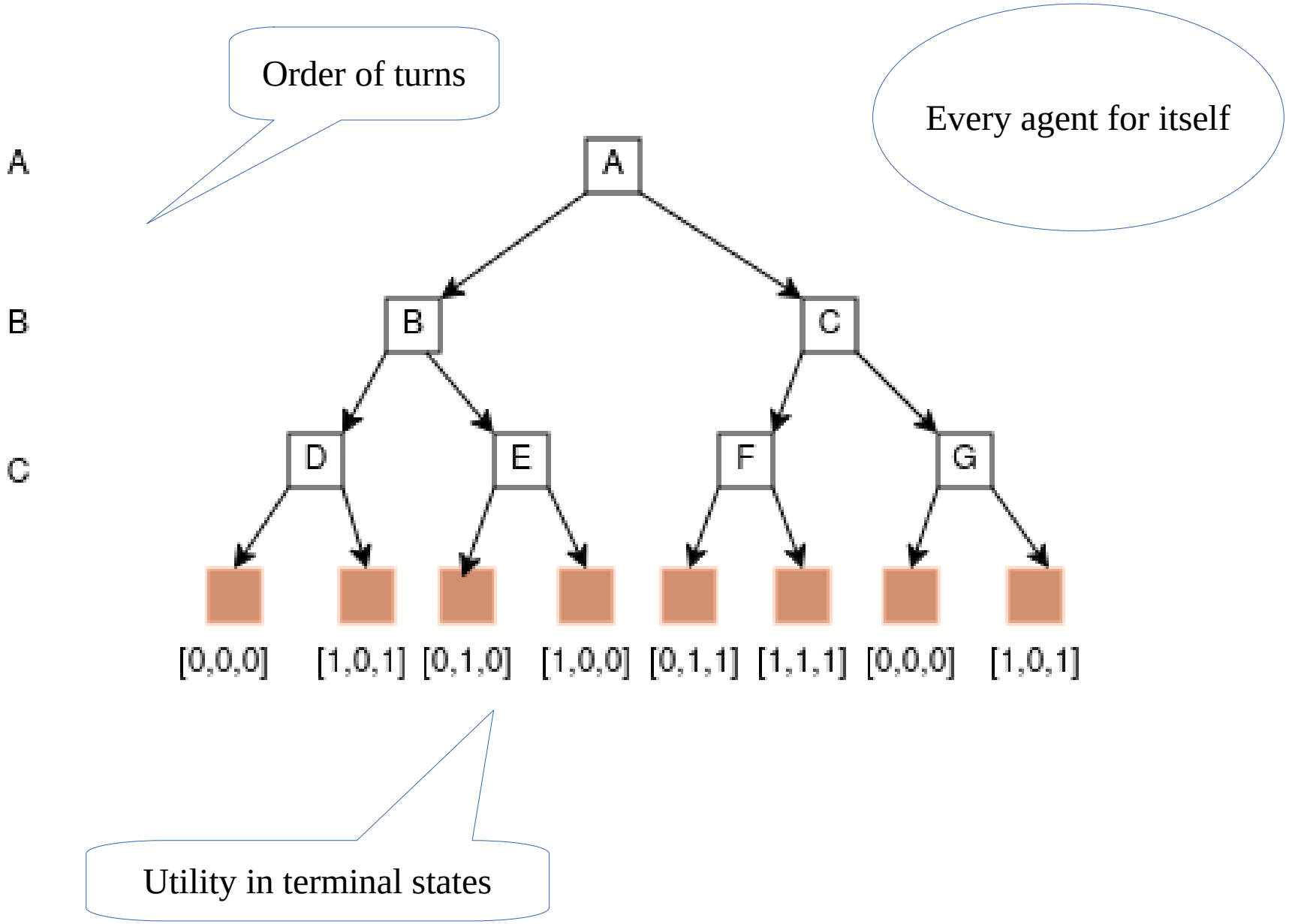
- Value of n is based on successor state:
  - Choose highest value for the player choosing at n



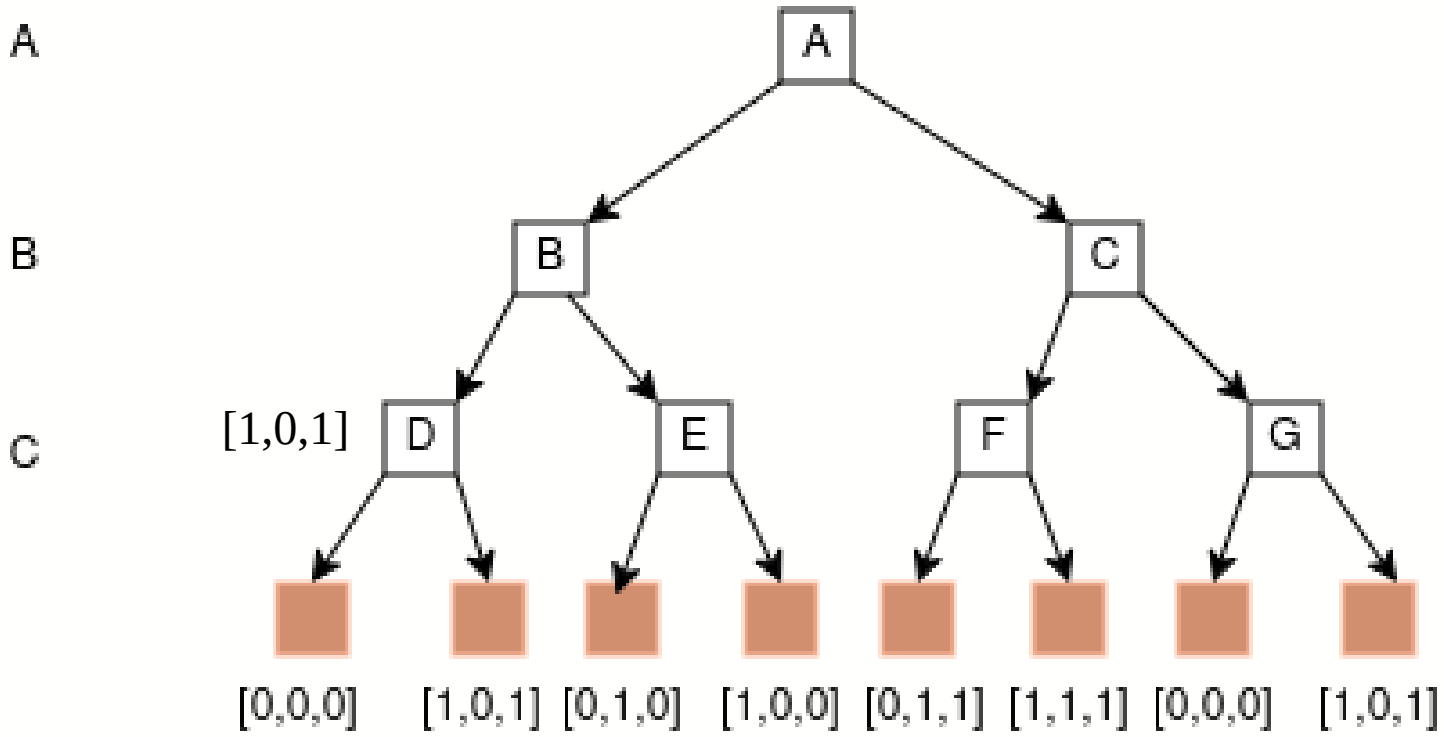
Multiplayer game via minimax

- Let’s first focus on competition: Usage of max assumes each agent is purely selfish
- Consider made up game:
  - There are three players. Each player can only make one move
  - Each player can either win/lose (not ‘zero sum’)
  - Each player is acting selfishly
  - We abstract away details regarding the states. Branching factor = 2. Depth = 4

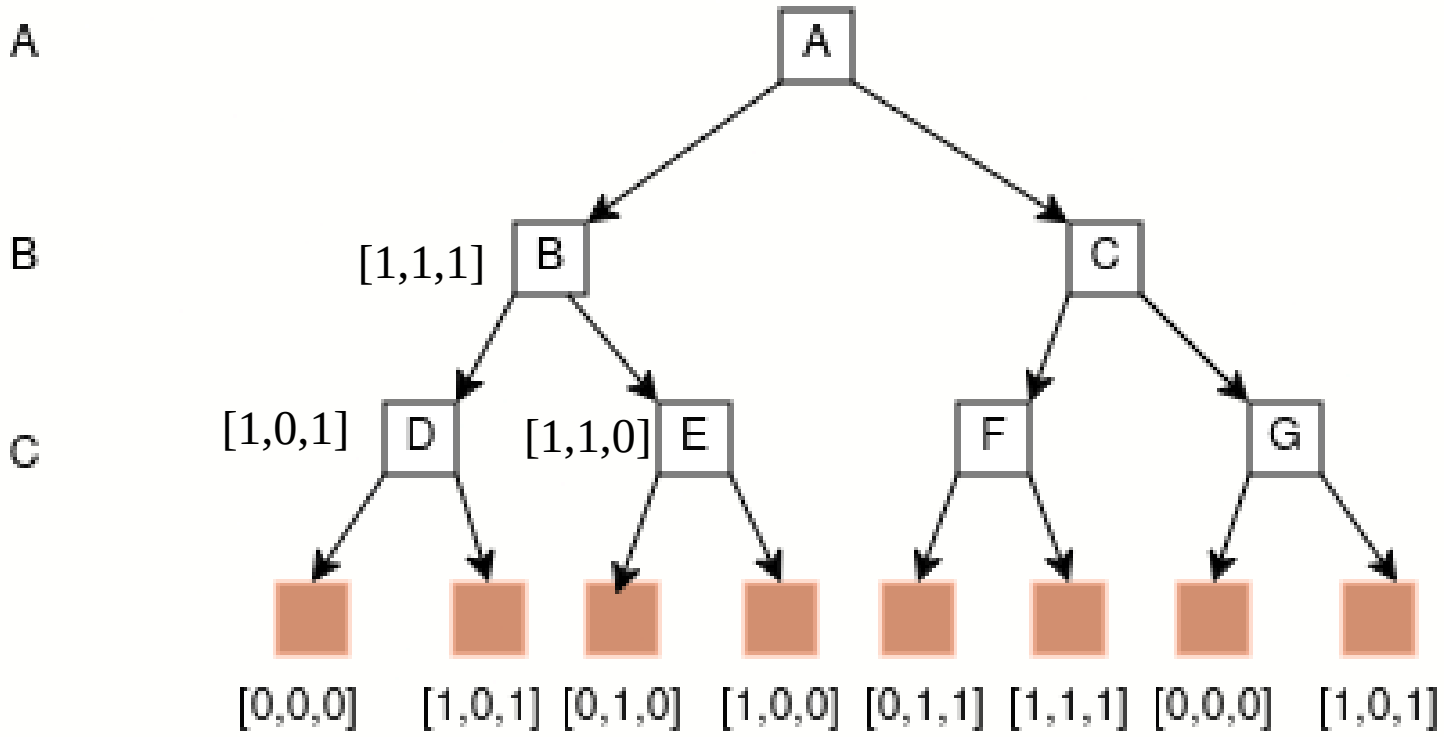
Multiplayer game via minimax



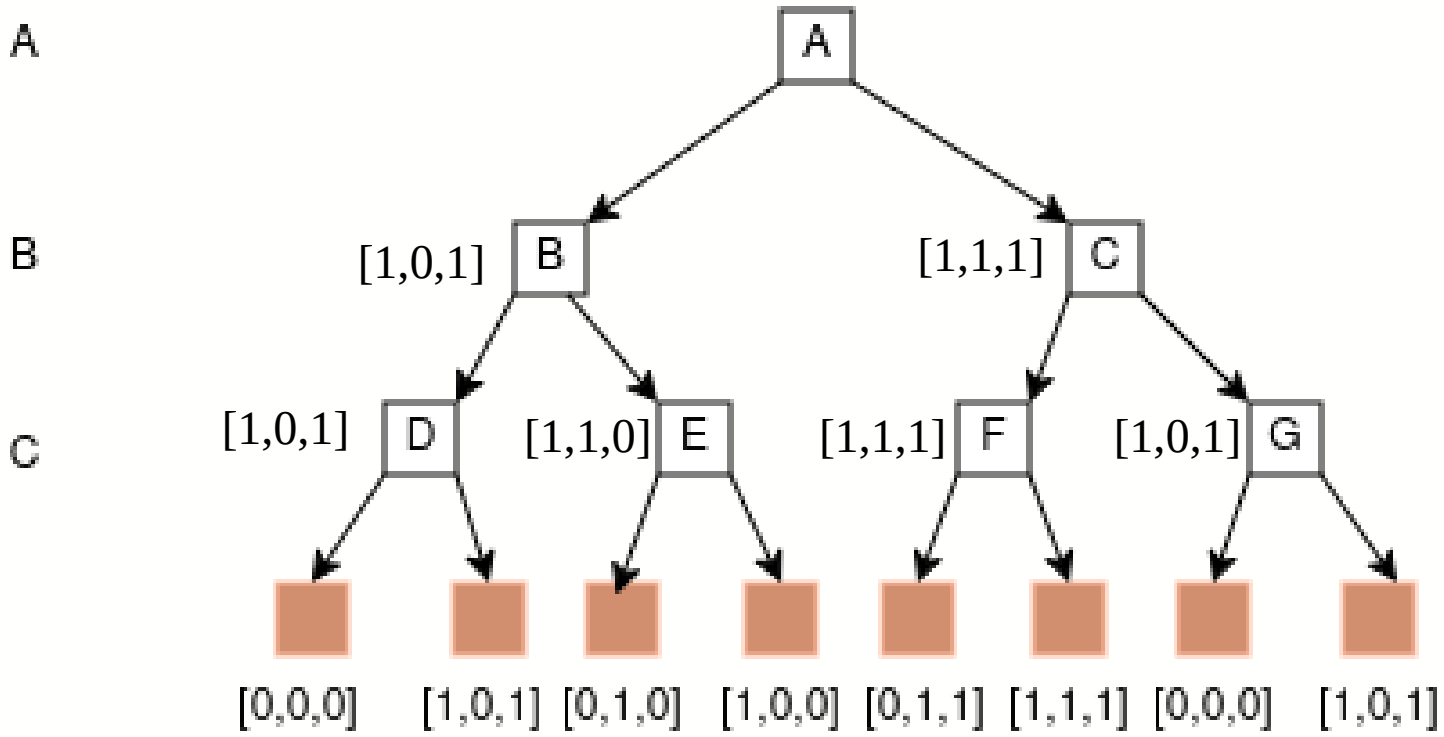
Multiplayer game via minimax



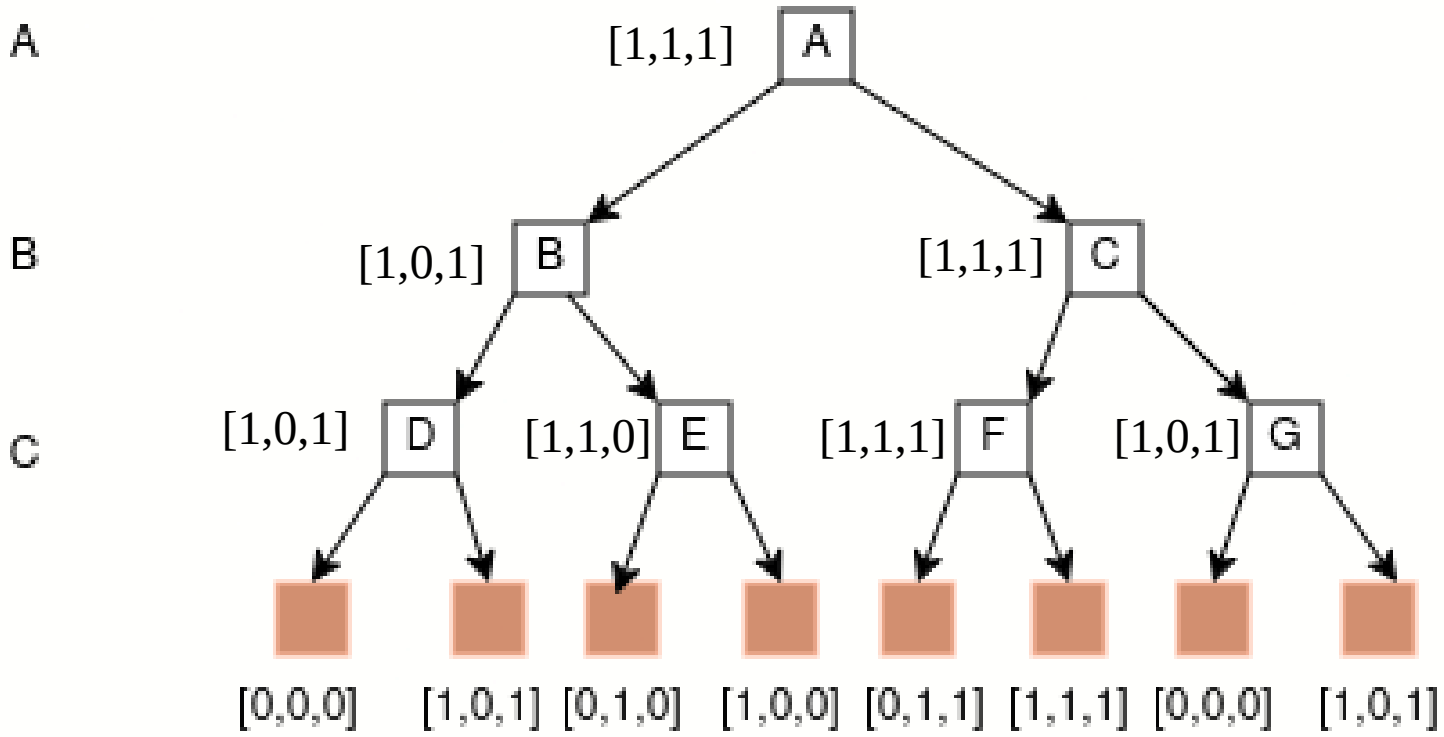
Multiplayer game via minimax



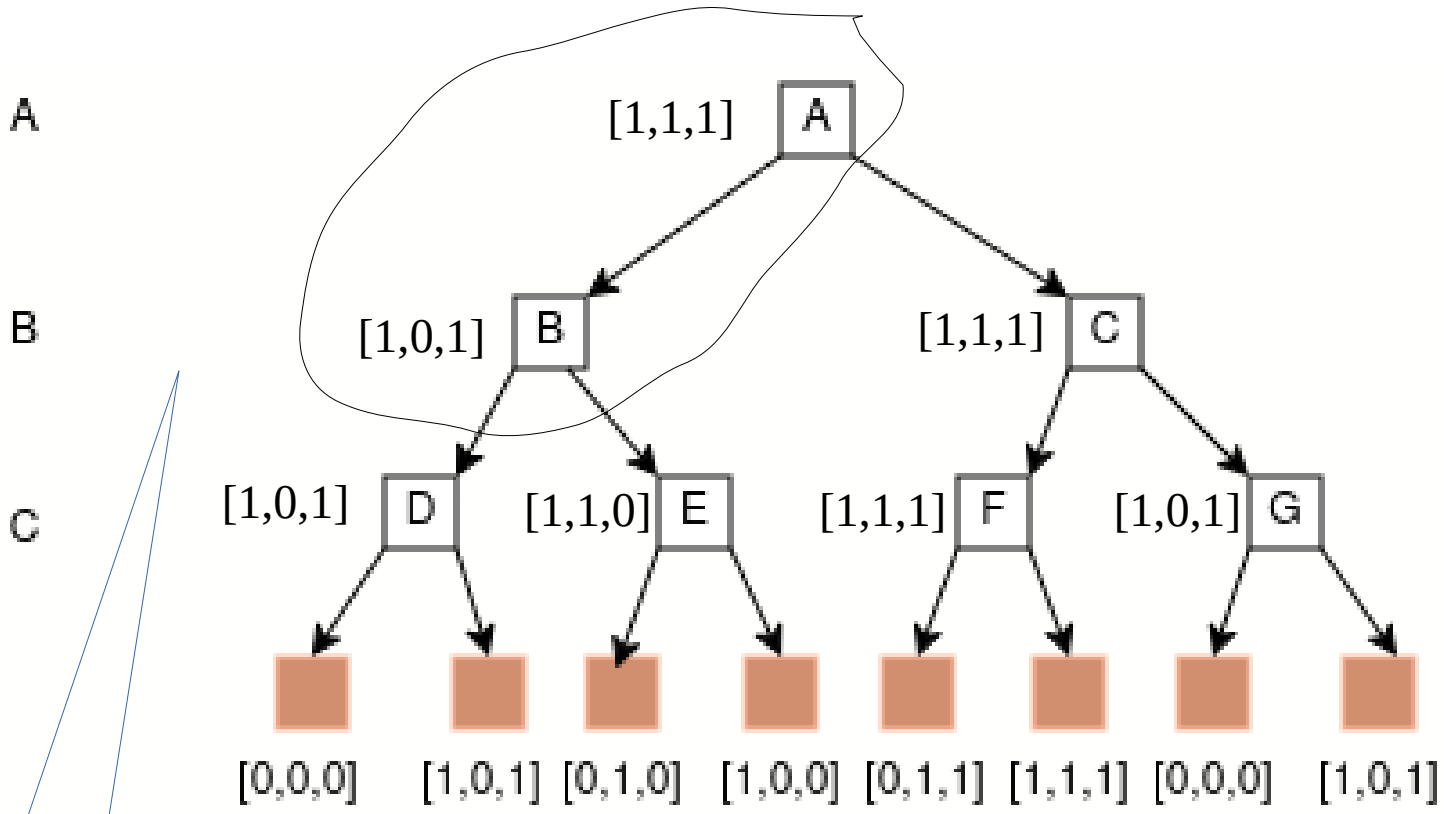
Multiplayer game via minimax



Multiplayer game via minimax

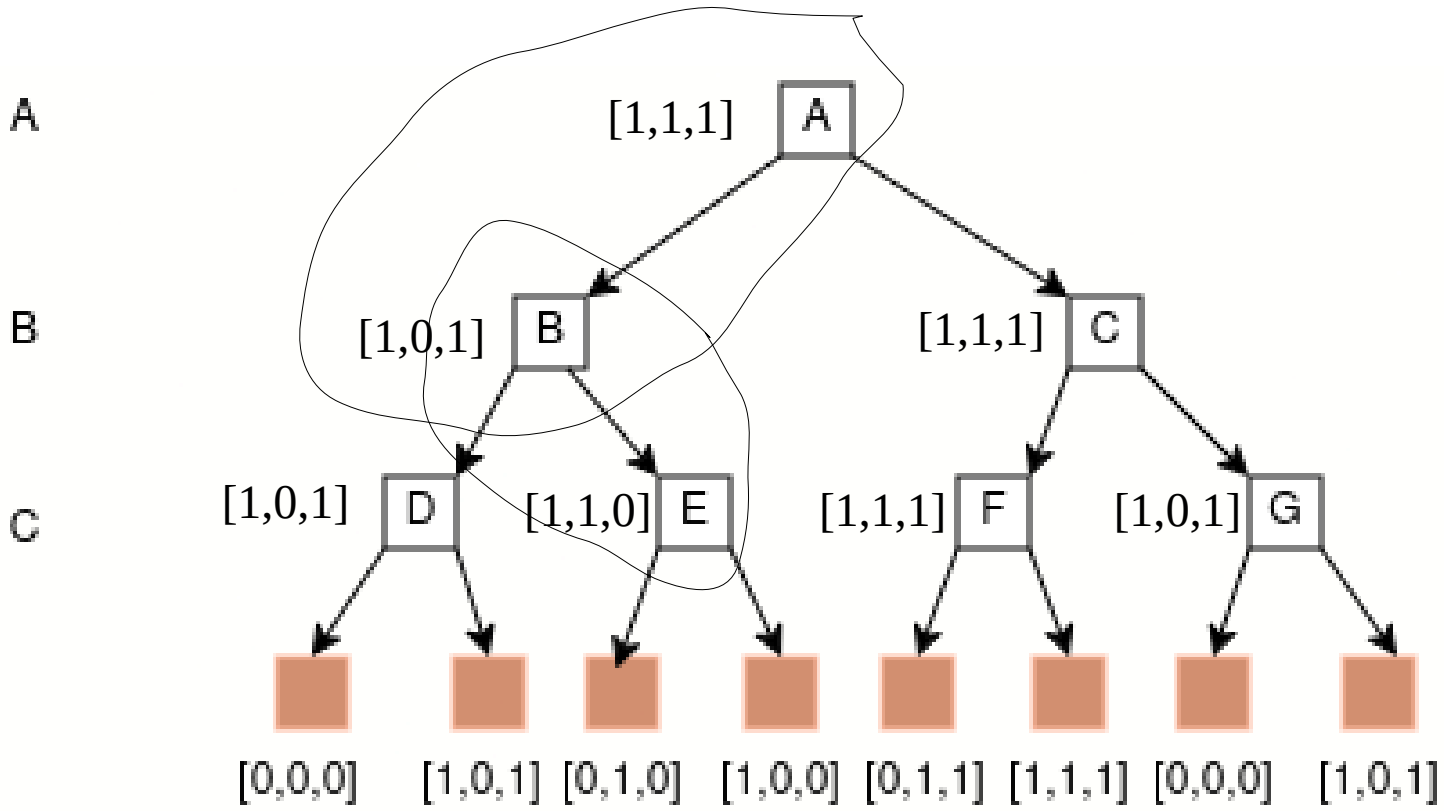


Multiplayer game via minimax

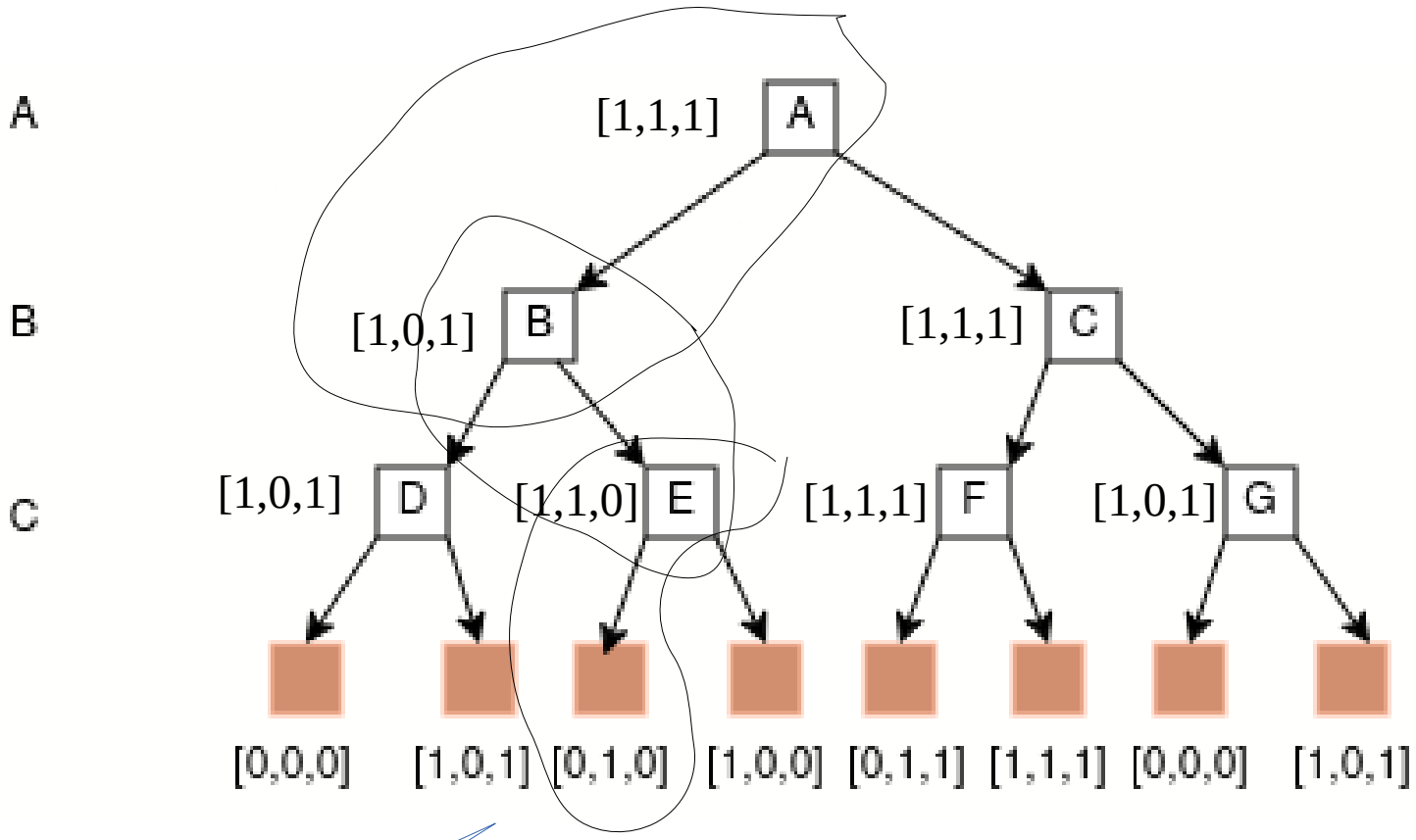


B is put in disadvantage (not intentional)

Multiplayer game via minimax

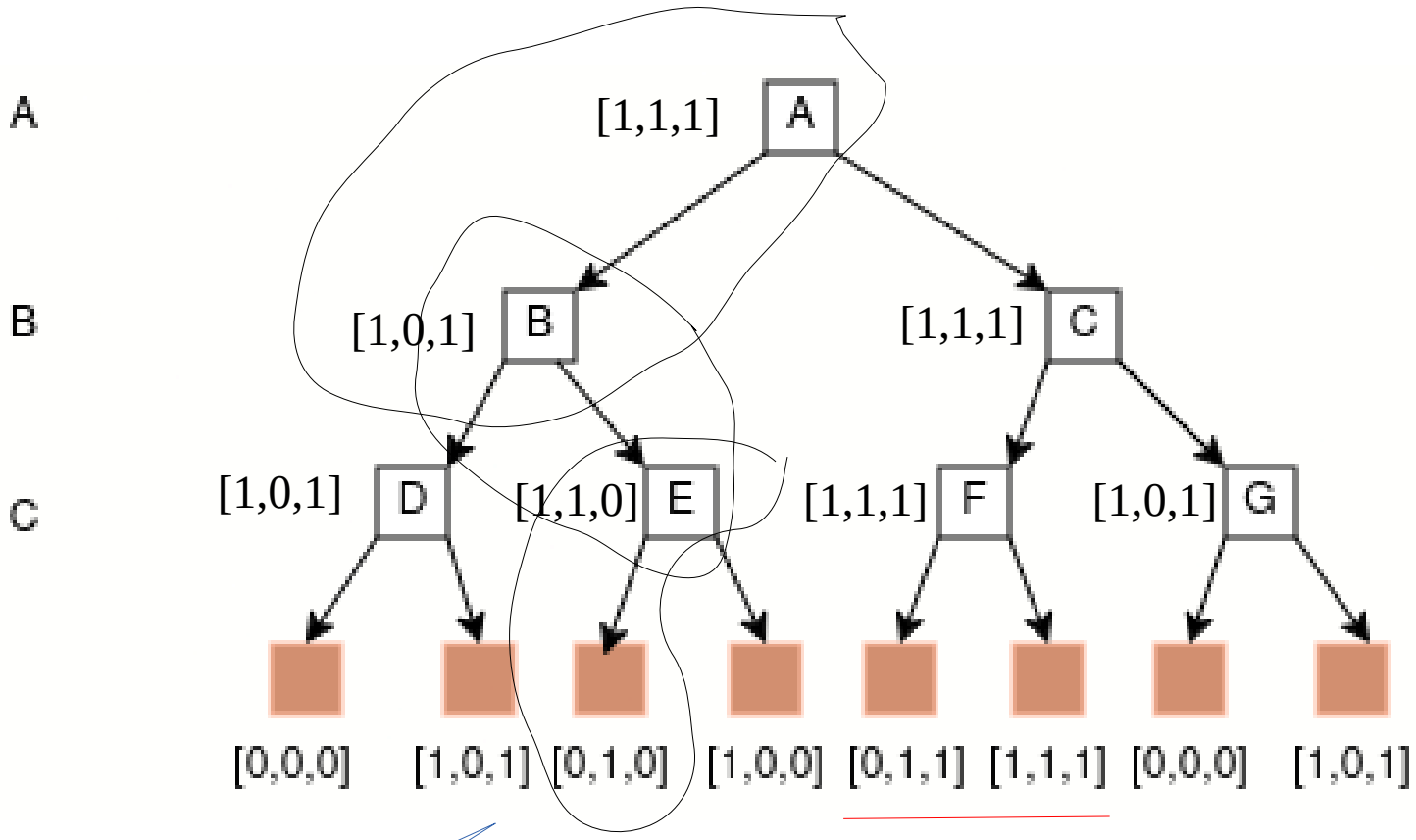


Multiplayer game via minimax



B ends up winning!

Multiplayer game via minimax



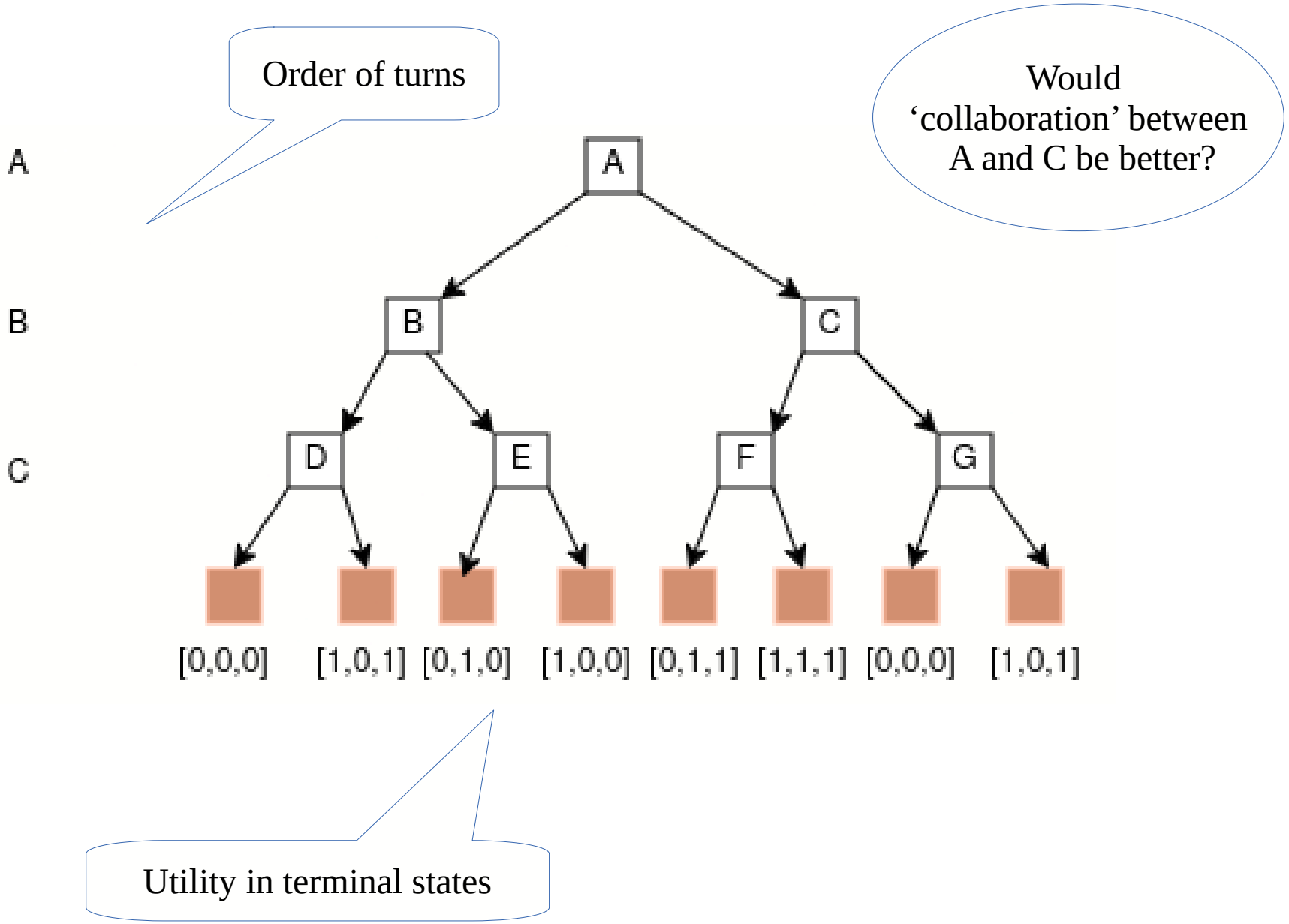
B ends up winning!

Multiplayer game via minimax

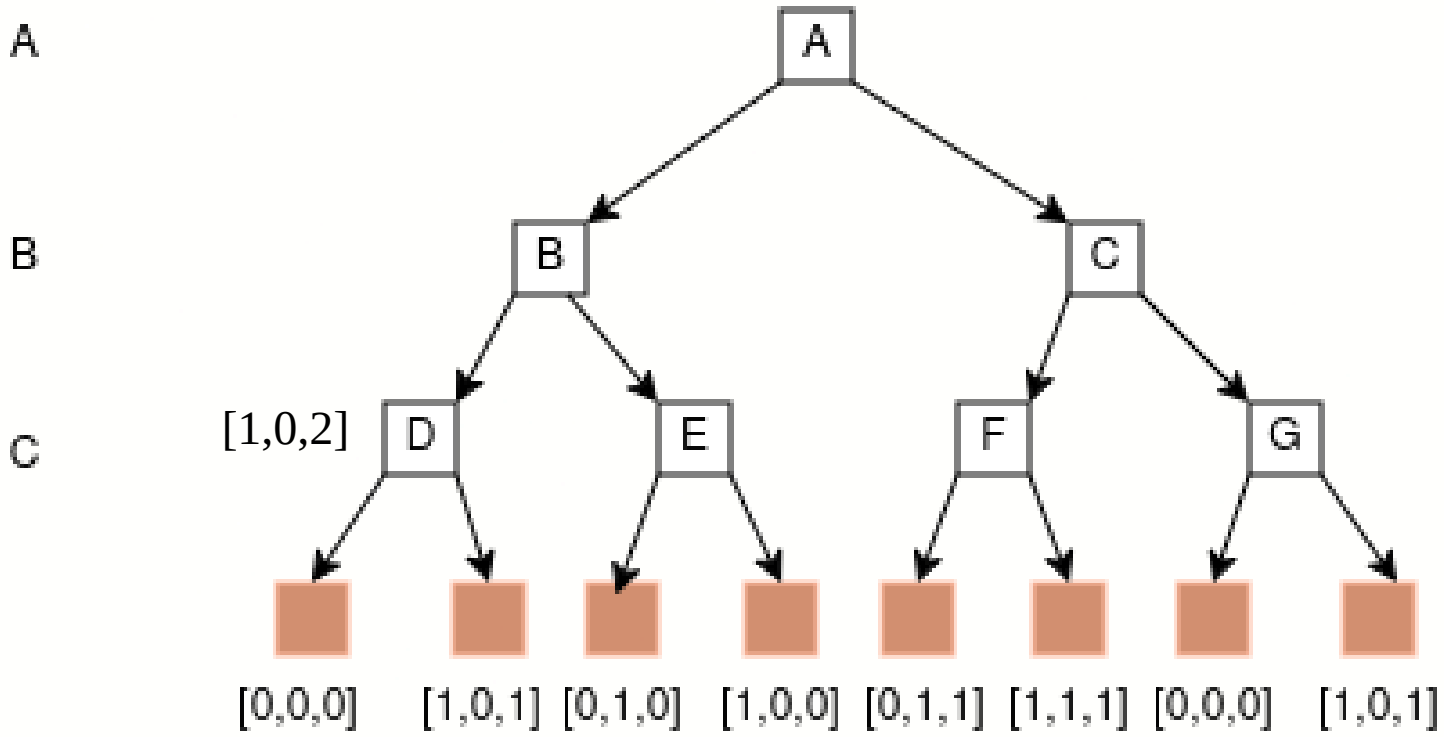
- ~~Competition: Usage of max assumes each agent is purely selfish~~
- Impact of cooperation: some agents work towards the same goal
  - i.e., one/more agent(s) prioritises the utility of more than one agent
  - Let's treat agent A and C as working together. Specifically, A helps C. Pay off from A 'injected' to C

$$minimax(s) = \begin{cases} utility(s) & terminal(s) \\ \max_{a \in actions} minimax(result(s,a)) & tomove(s) = A/B \\ \max_{a \in actions} minimax(result(s,a)) + V_A & tomove(s) = C \end{cases}$$

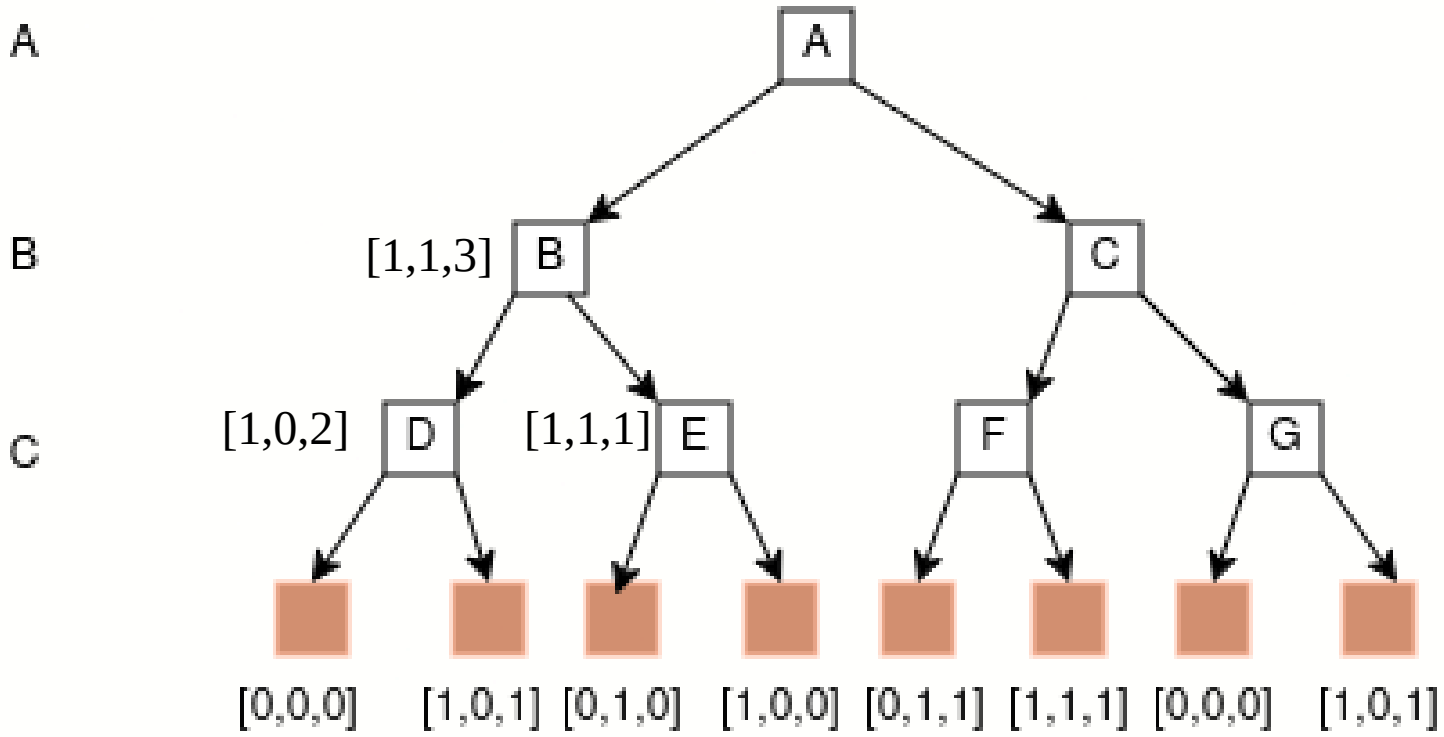
Multiplayer game via minimax



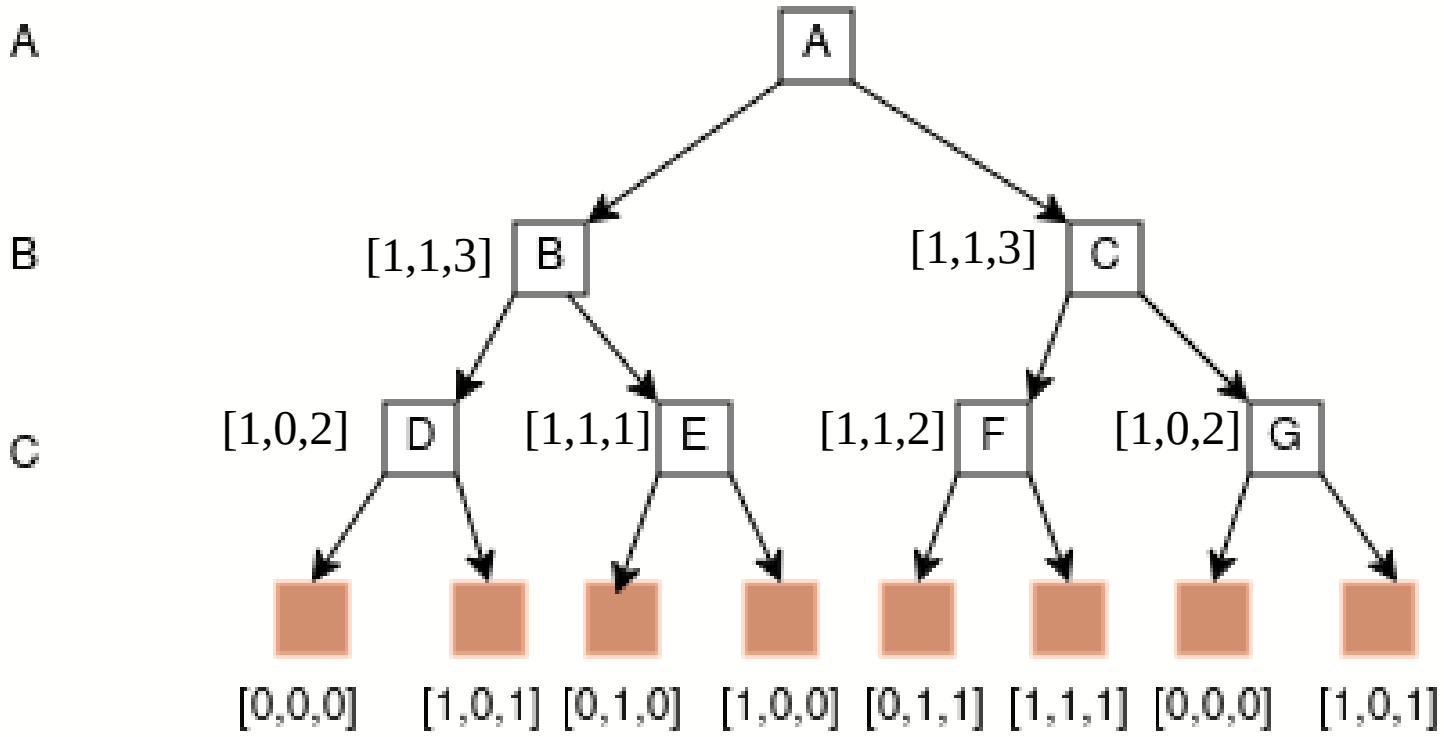
Multiplayer game via minimax



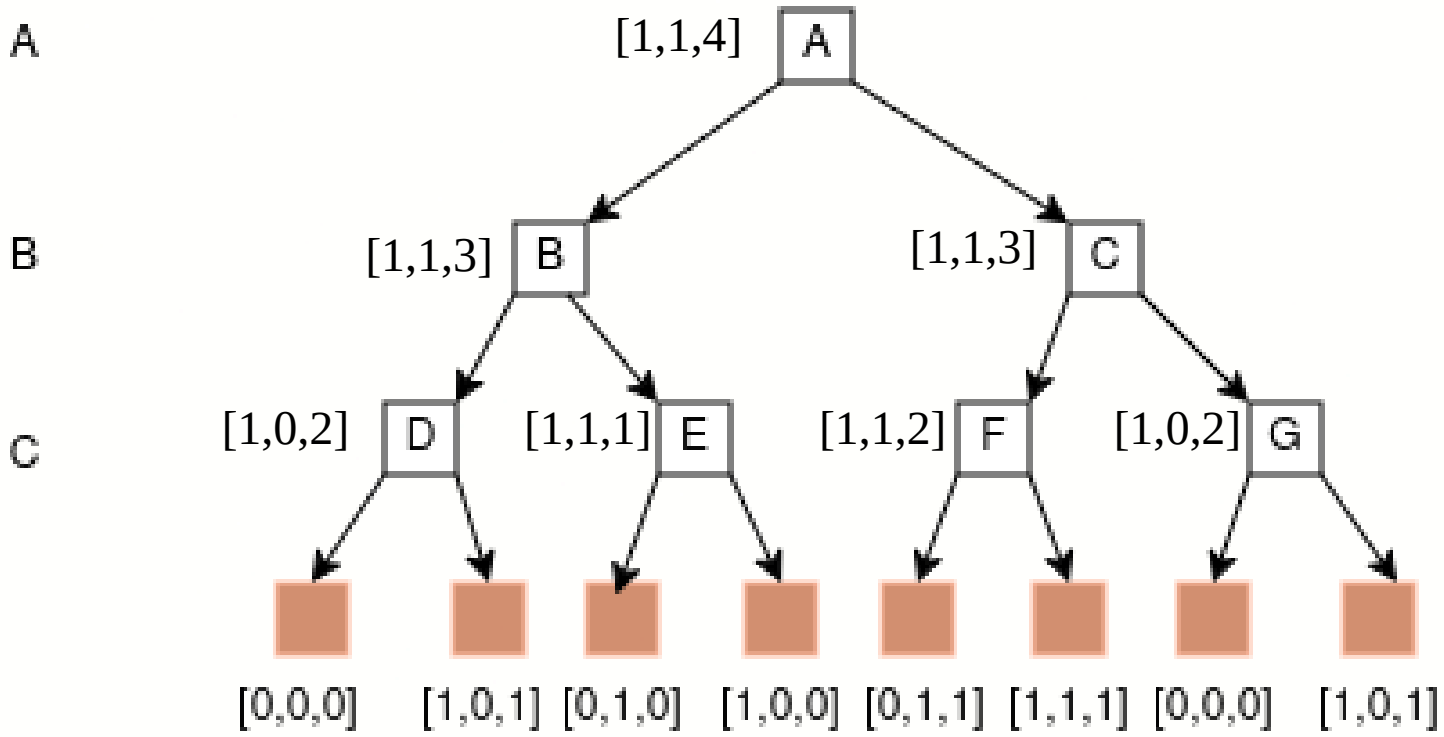
Multiplayer game via minimax



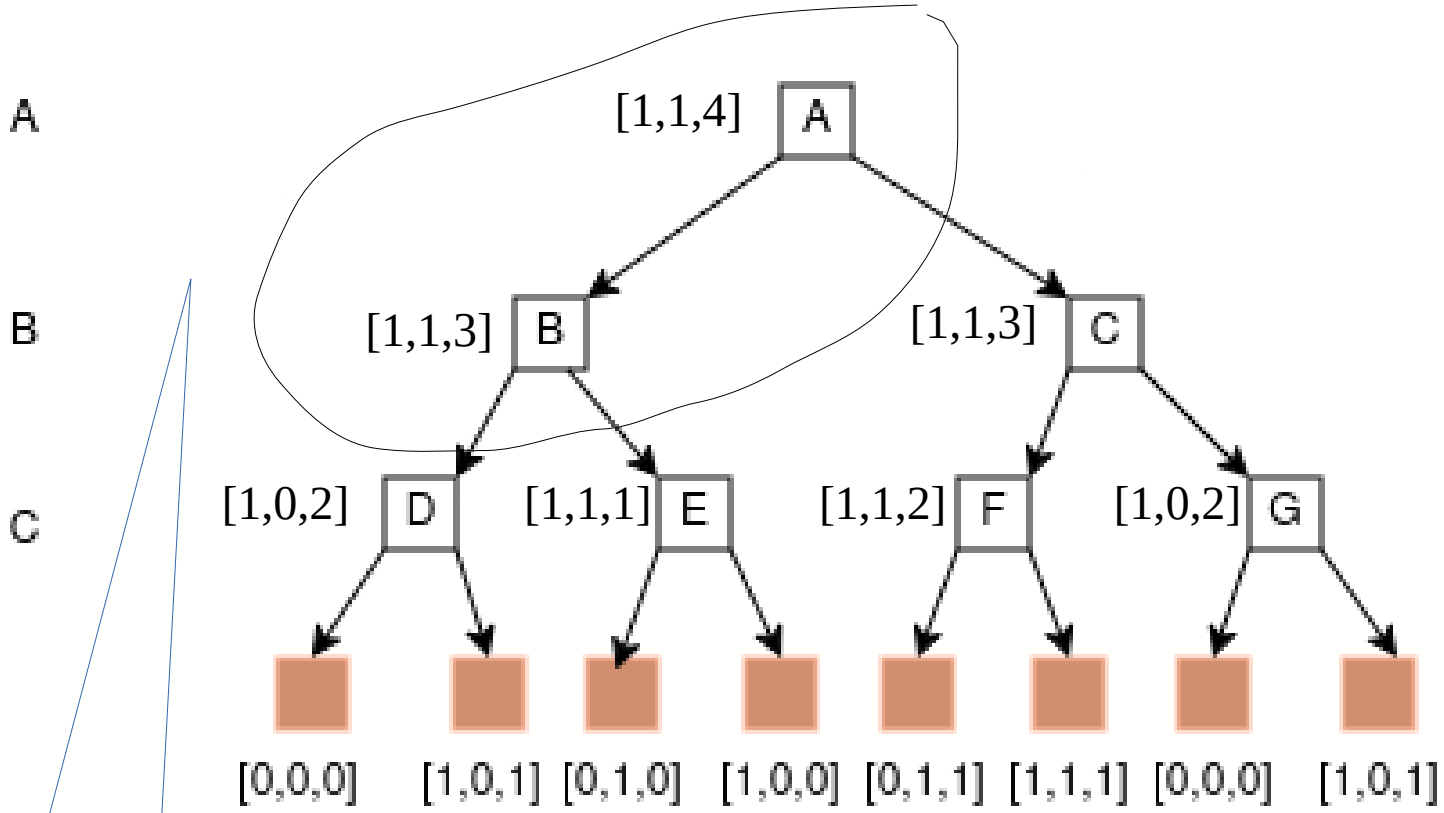
Multiplayer game via minimax



Multiplayer game via minimax

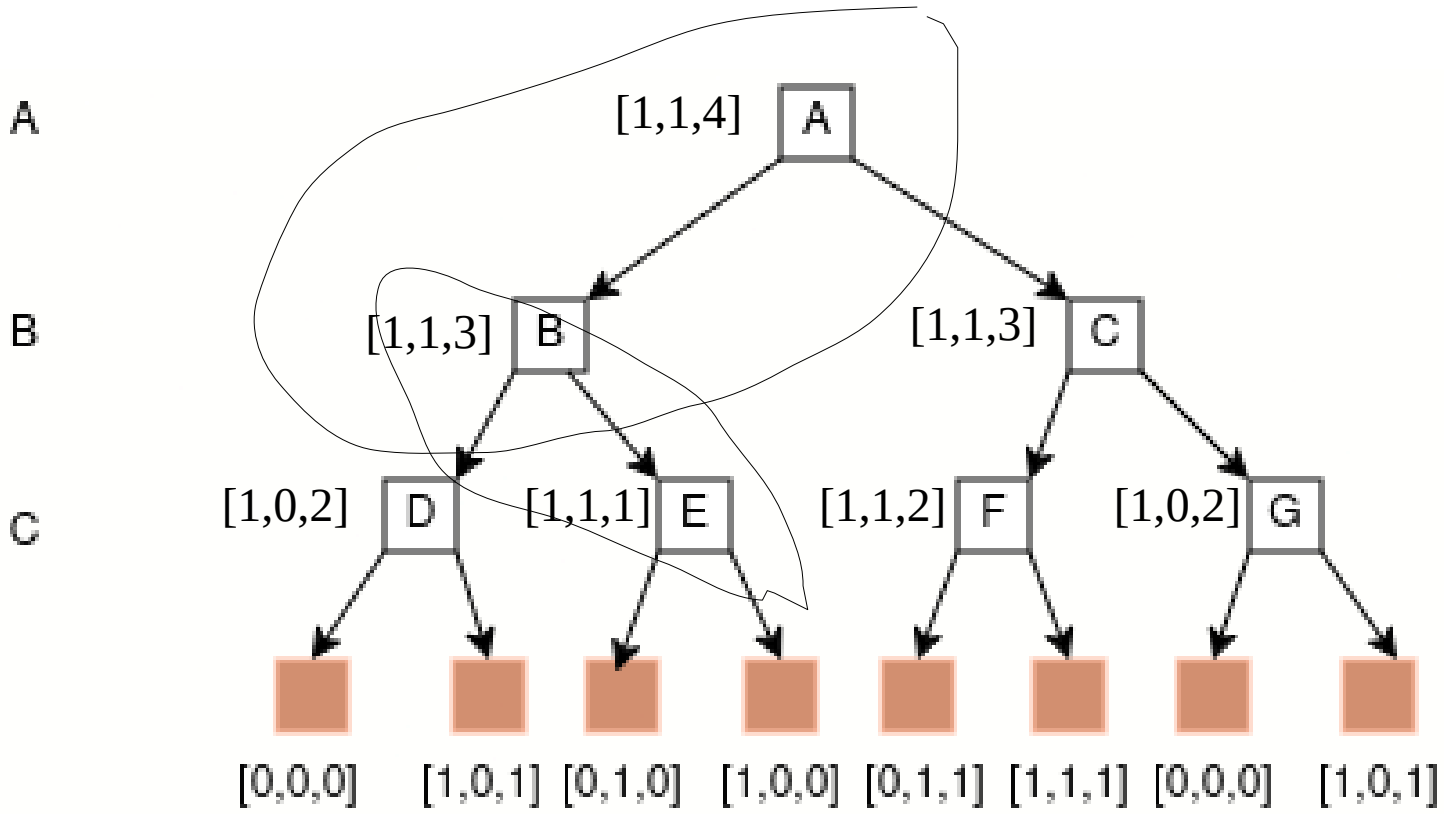


Multiplayer game via minimax

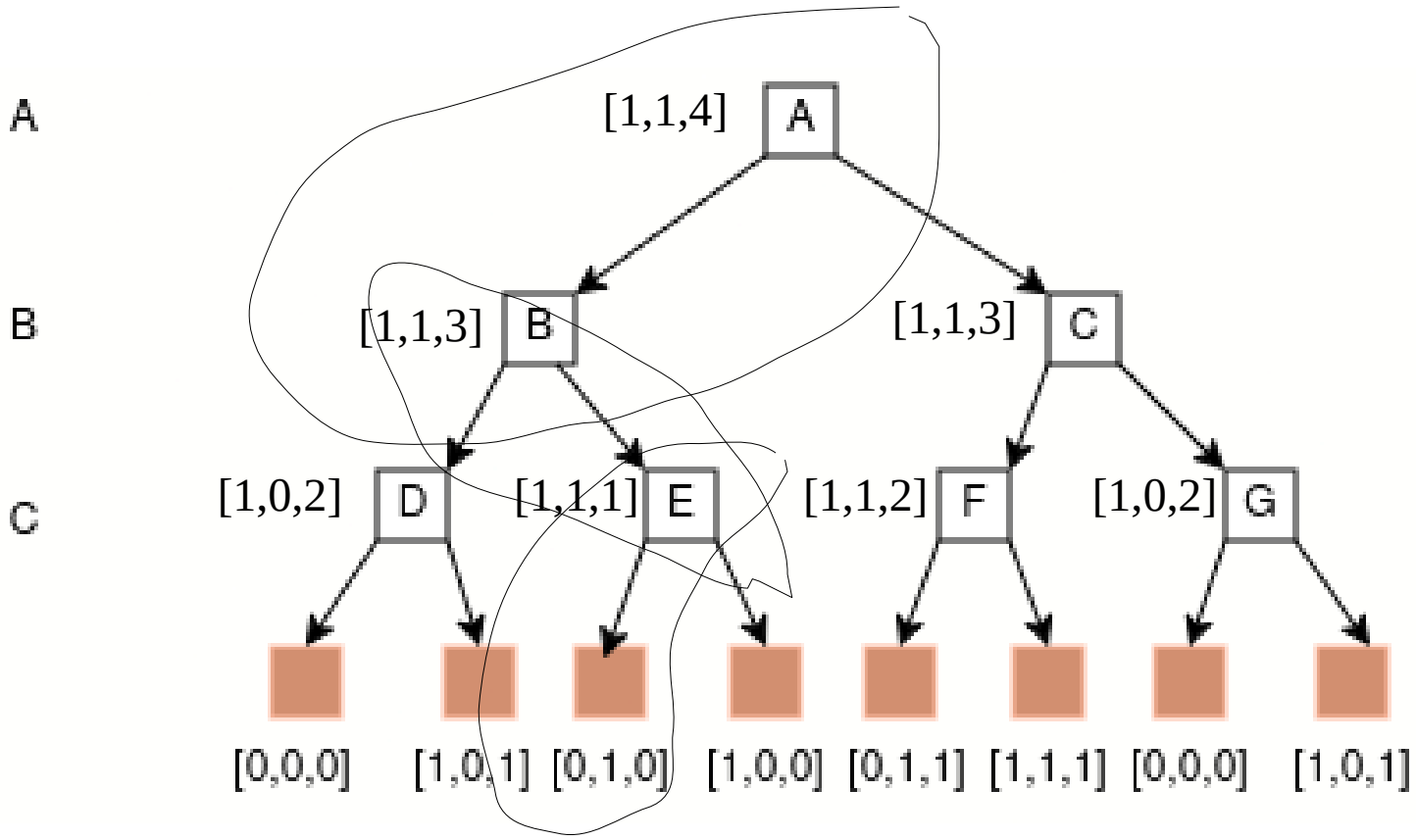


A's decisions are not Influenced by C

Multiplayer game via minimax



Multiplayer game via minimax



B still wins, despite Cooperation!

Multiplayer game via minimax

- How to keep the ‘zero-sum’ nature of the game?
  - If A/C win, then B needs to lose!
  - Any gain by A/C needs to be lost by B
- ‘Zero-sum’ has a colloquial interpretation and formal one
  - Sum of utilities at terminal states needs to be zero.
  - You can generalise concept to K-sum. Then zero sum equiv. Situation where k = 0.
- Total value is fixed. Its gained by one agent at the expense of the other
  - Either A/C wins (i.e, sum = 1) or B wins (i.e., value = 1)

Multiplayer game via minimax

- Consider a K-sum problem thus at terminal states:
  - $V_A + V_B + V_C = K$
- At terminal state, we rely on utility function
  - minimax(s) = utility(s)
- minimax(s) = max{minimax(result(s, a))} for all available actions a if tomove(s) = A

Agent A is still acting selfishly

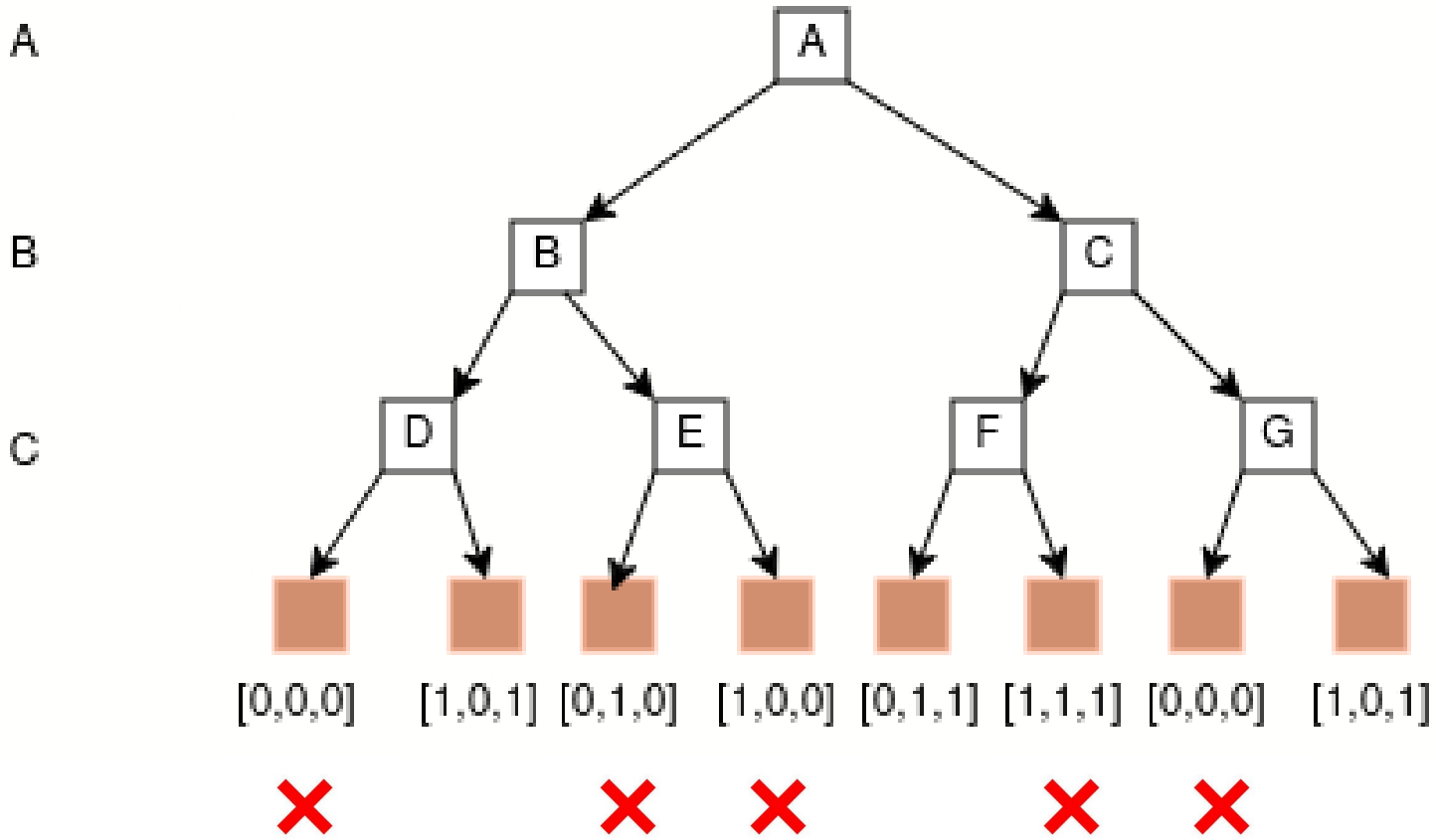
Multiplayer game via minimax: k-sum

- Consider a K-sum problem thus at terminal states:
  - $V_A + V_B + V_C = K$
- At terminal state, we rely on utility function
  - minimax(s) = utility(s)

C's cooperation with A
Is at the expense of B

- minimax(s) = max{minimax(result(s, a))} for all available actions a if tomove(s) = A
- minimax(s) = max{minimax(result(s, a)) - V_A} for all available actions a if tomove(s) = B
- minimax(s) = max{minimax(result(s, a)) + V_A} for all available actions a if tomove(s) = C

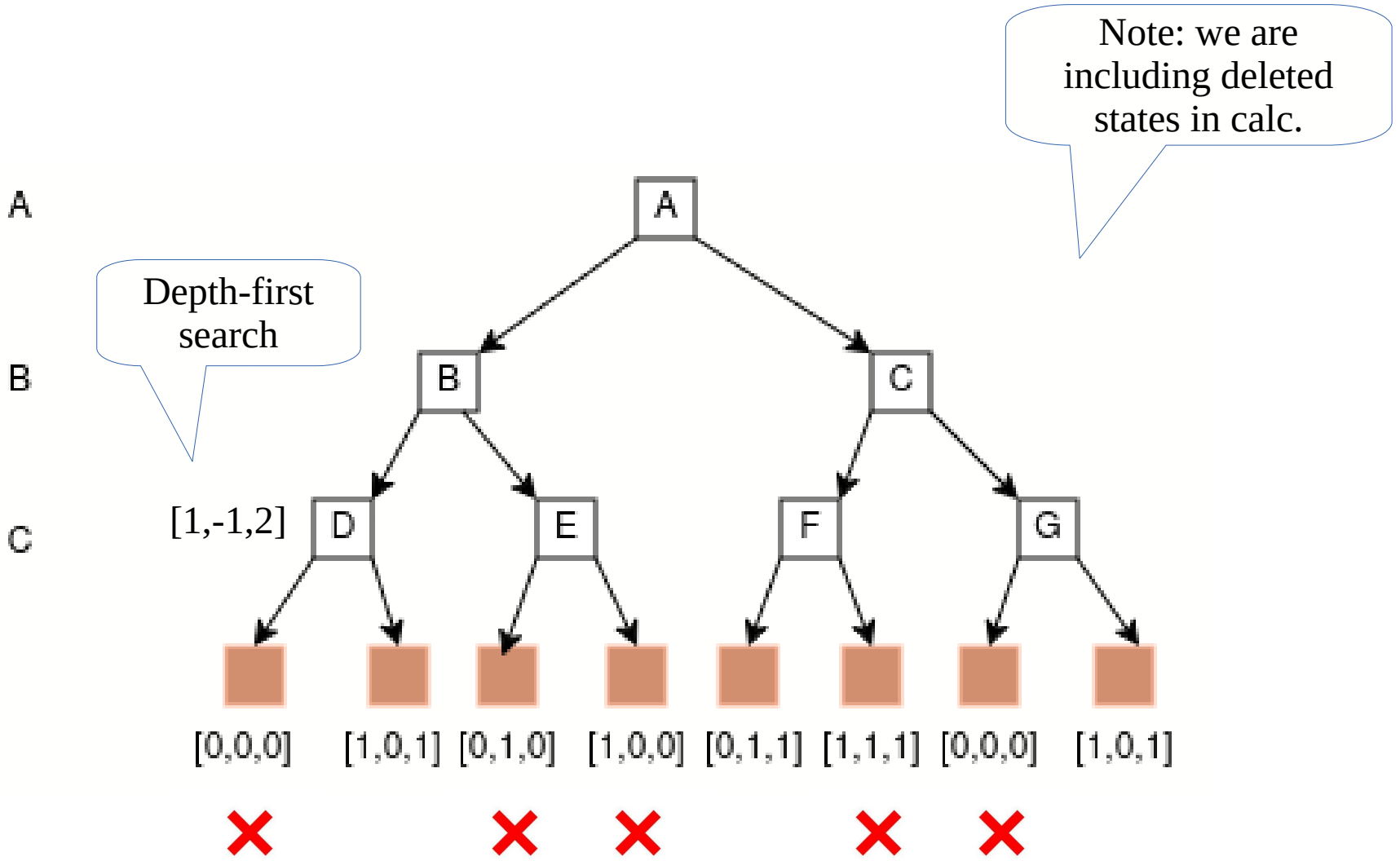
Multiplayer game via minimax: k-sum



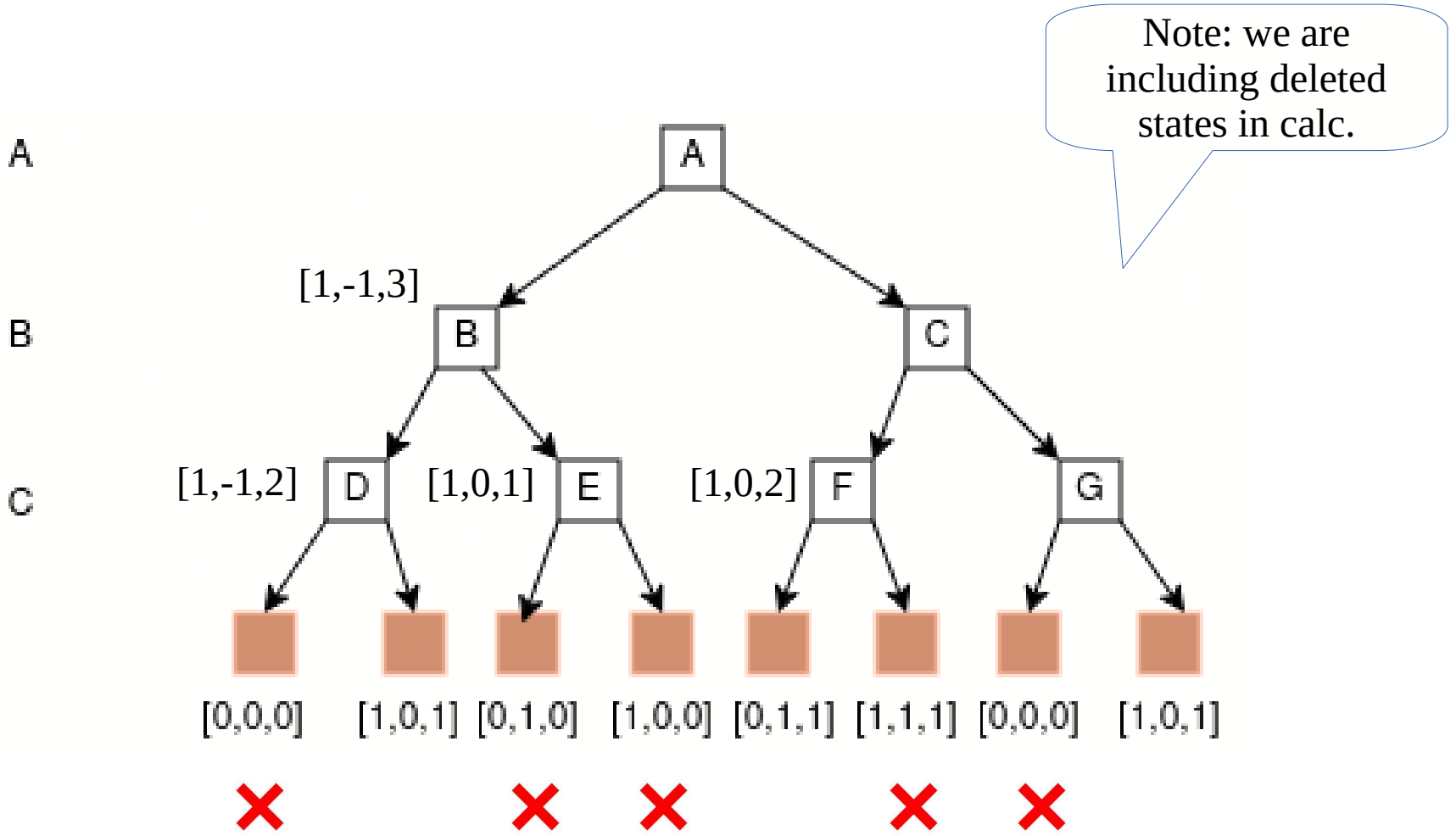
Terminal
States do not satisfy
Our k-sum condition

Note. K = 2

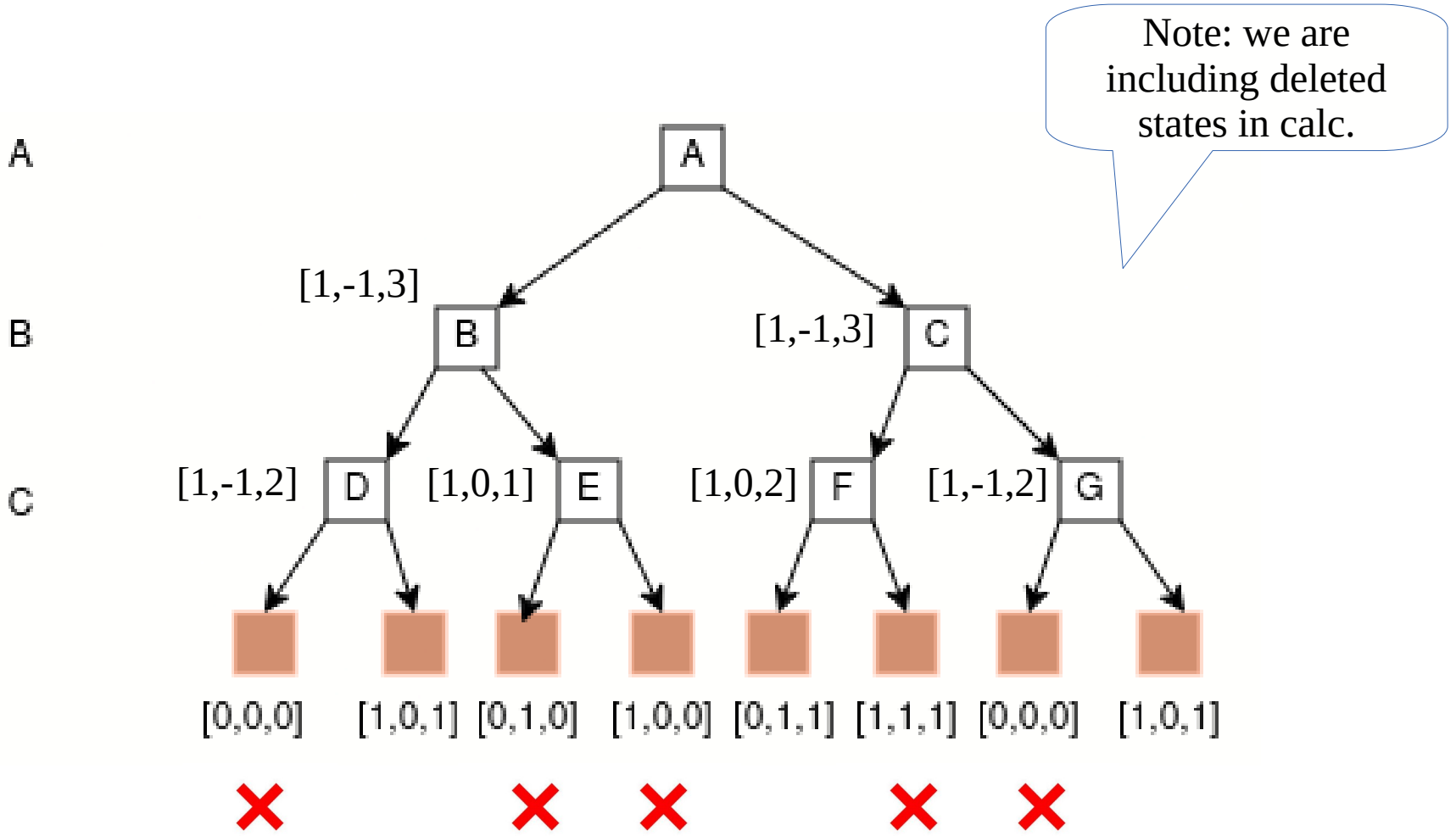
Multiplayer game via minimax: k-sum



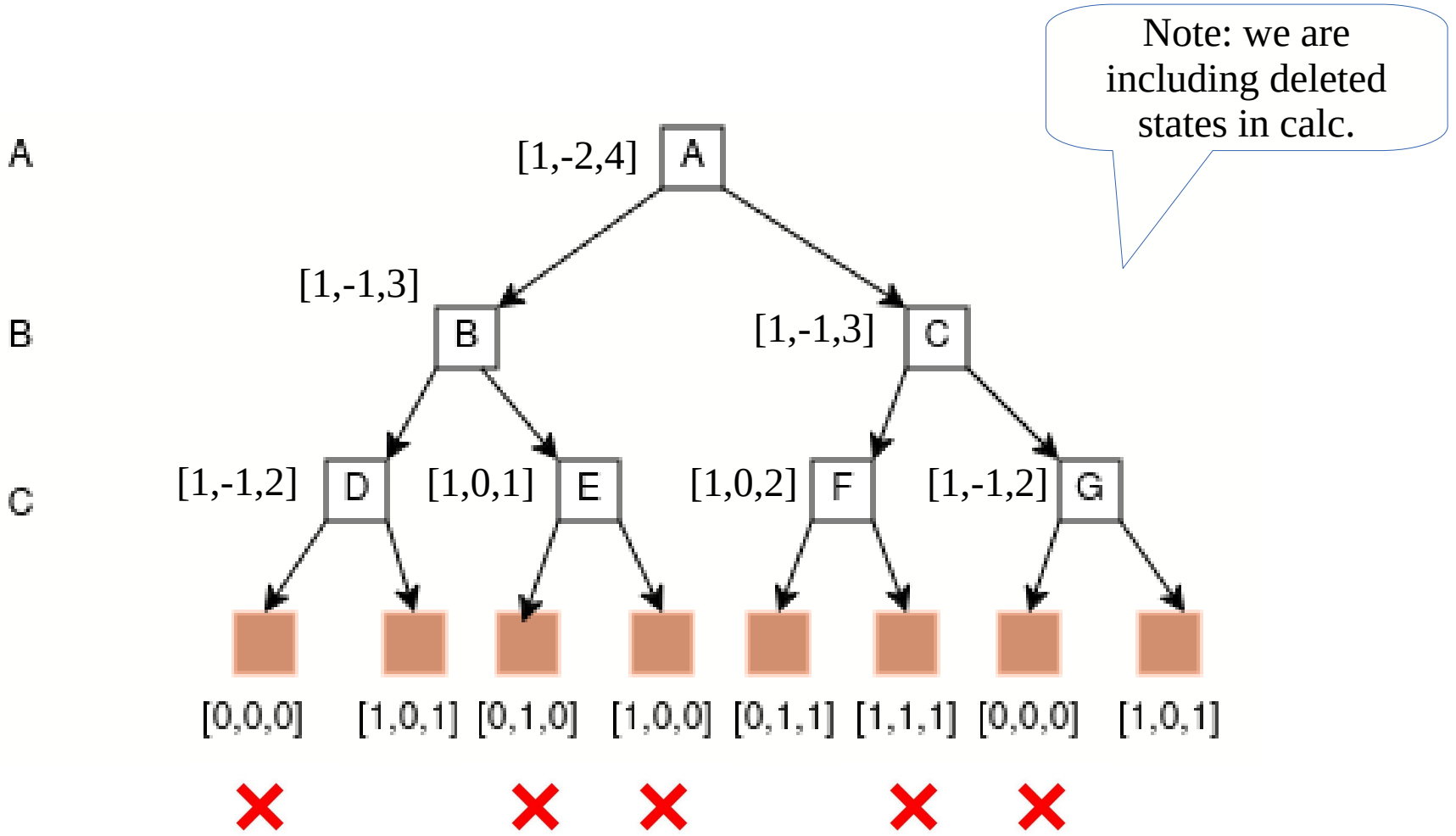
Multiplayer game via minimax: k-sum



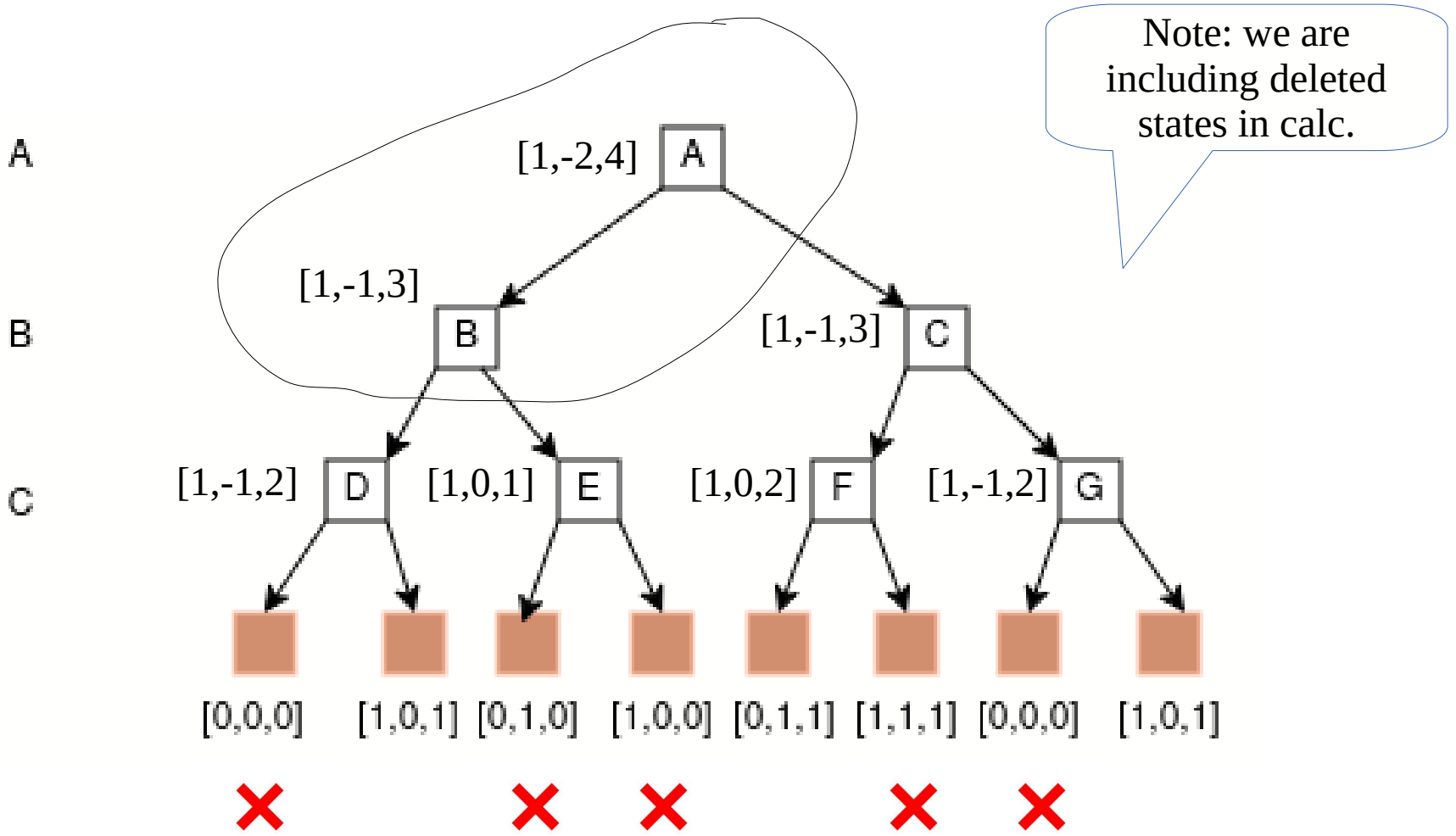
Multiplayer game via minimax: k-sum



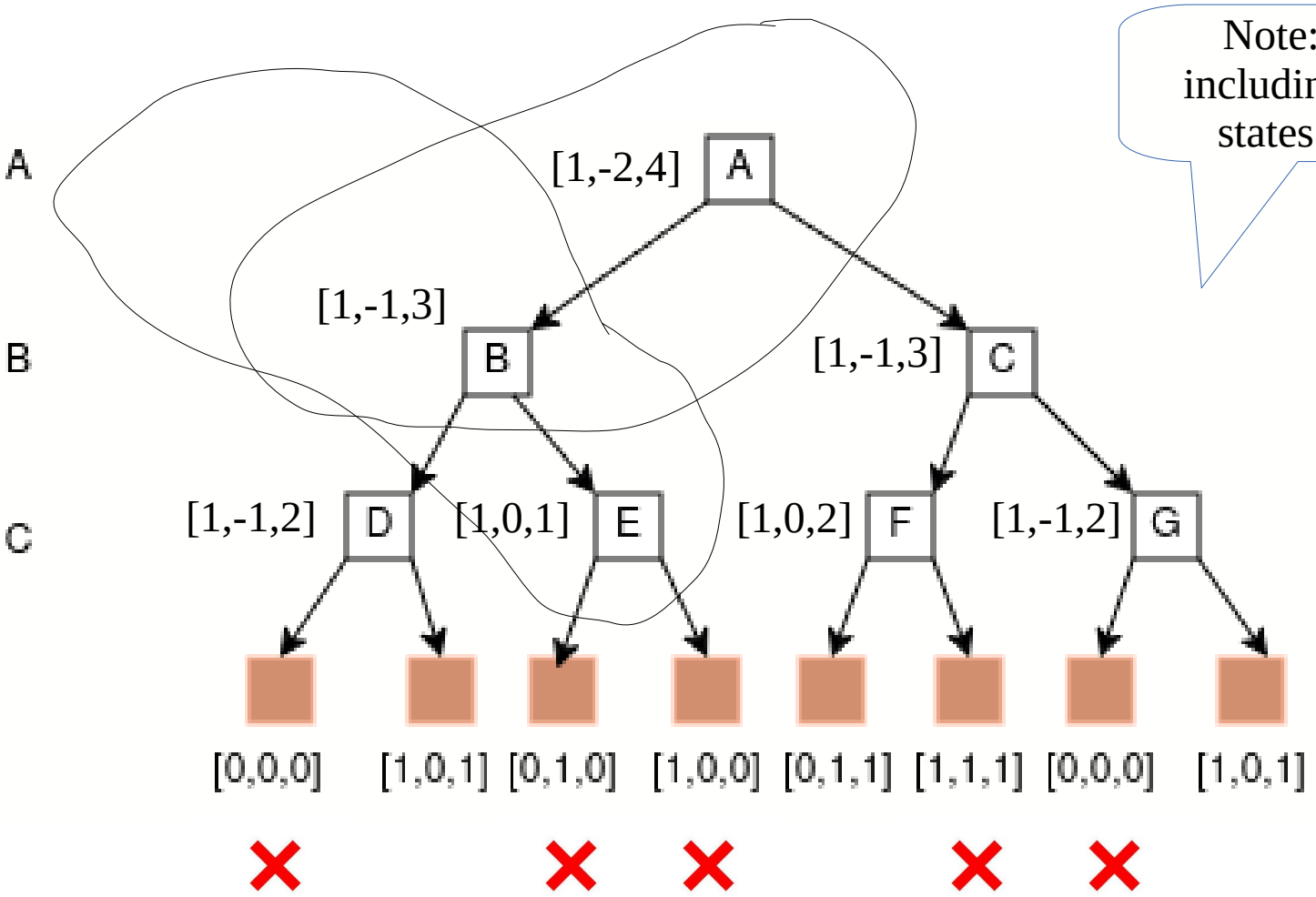
Multiplayer game via minimax: k-sum



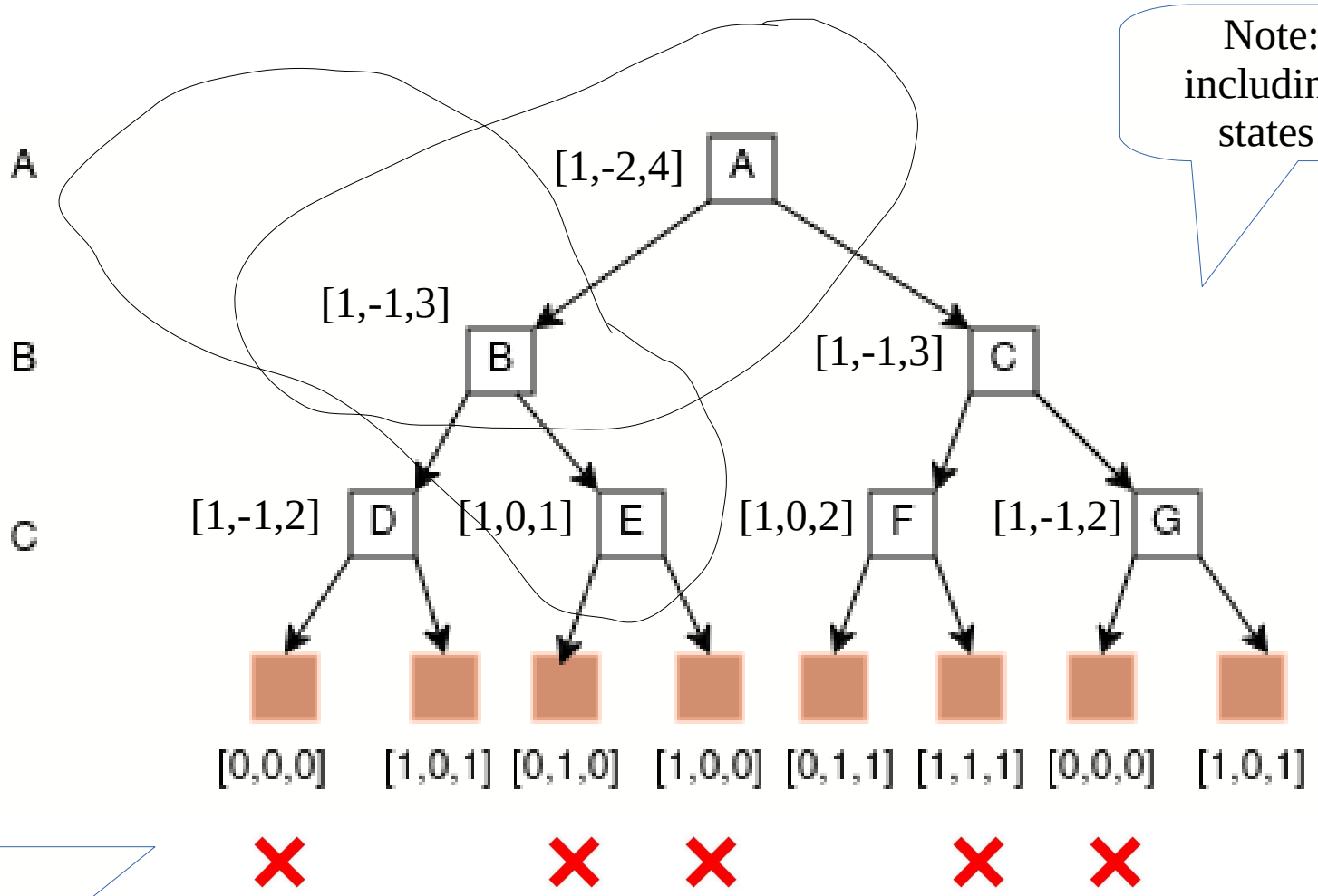
Multiplayer game via minimax: k-sum



Multiplayer game via minimax: k-sum



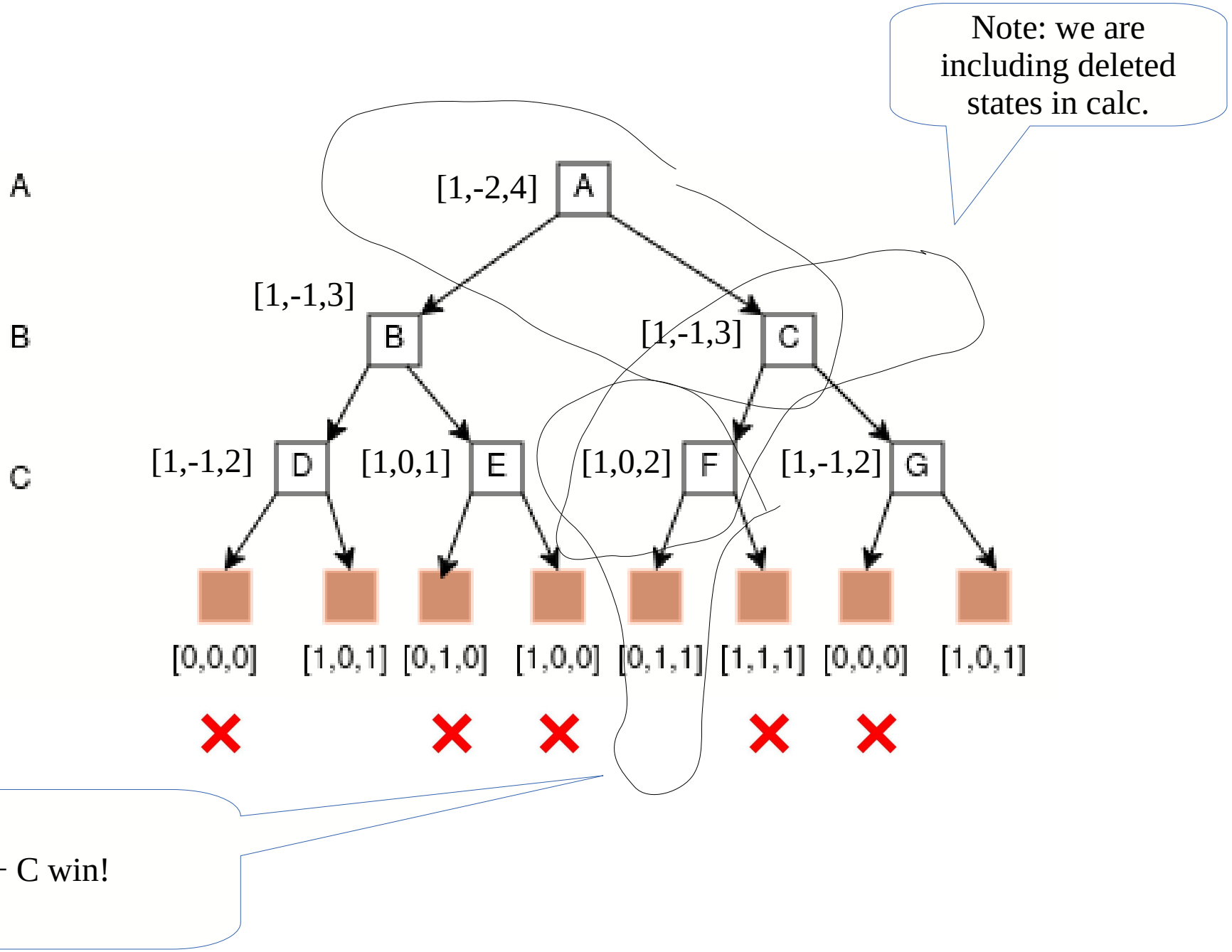
Multiplayer game via minimax: k-sum



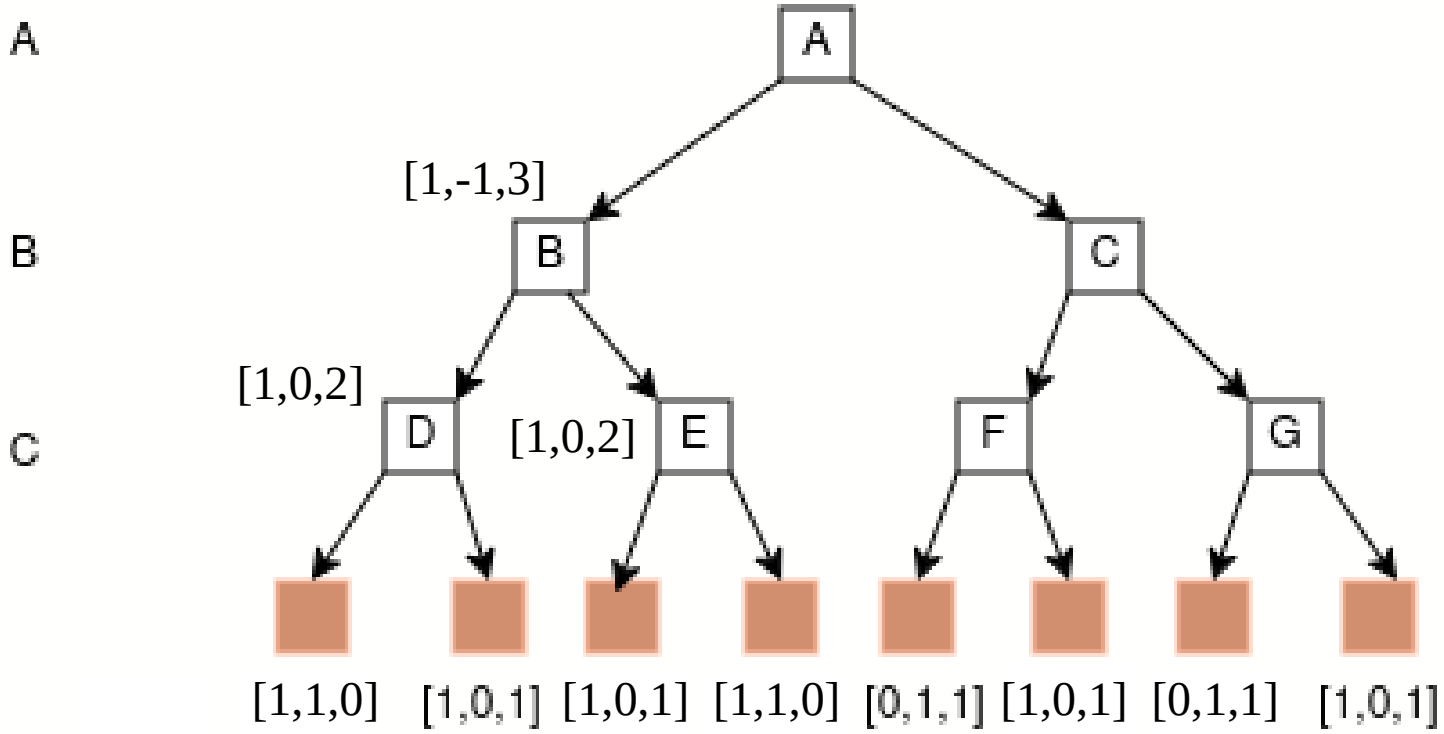
Note: we are including deleted states in calc.

Stalemate!

Multiplayer game via minimax: k-sum

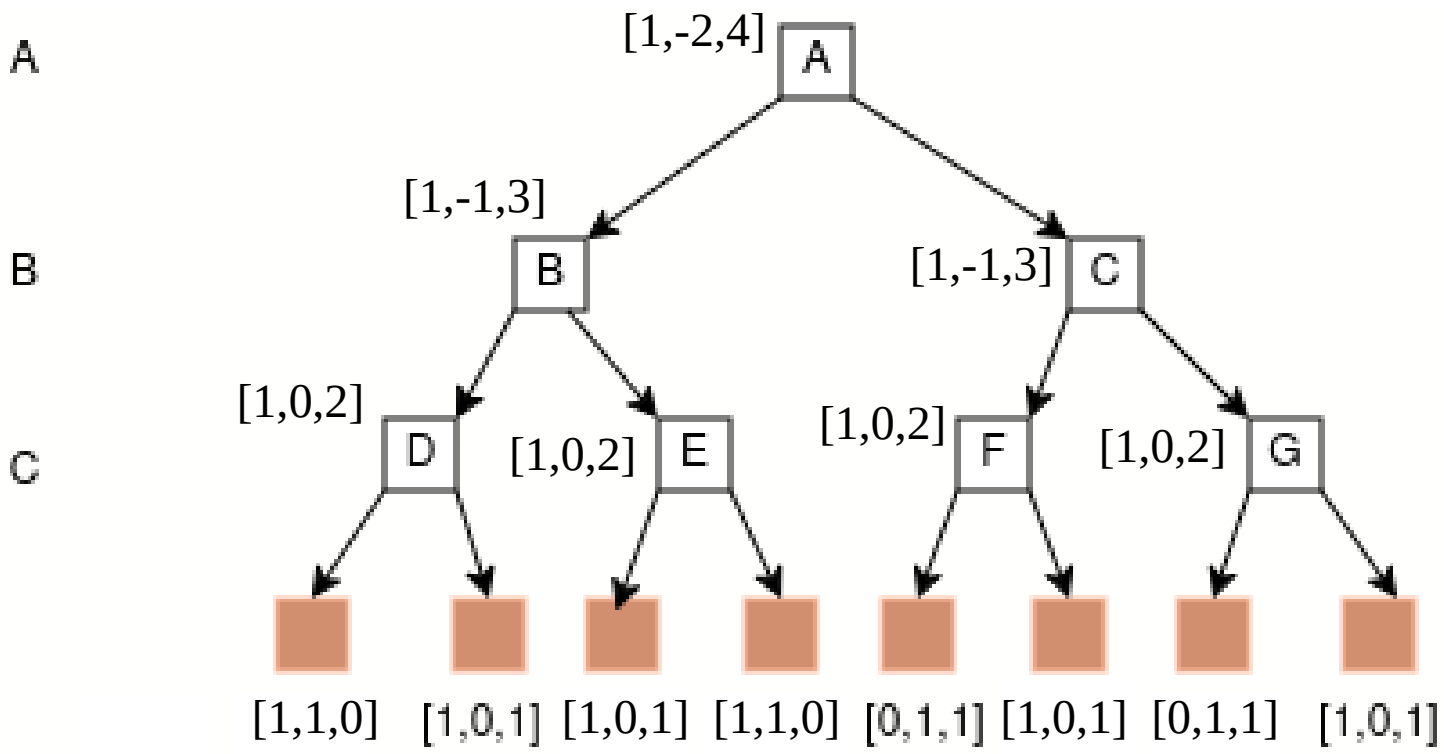


Multiplayer game via minimax: k-sum

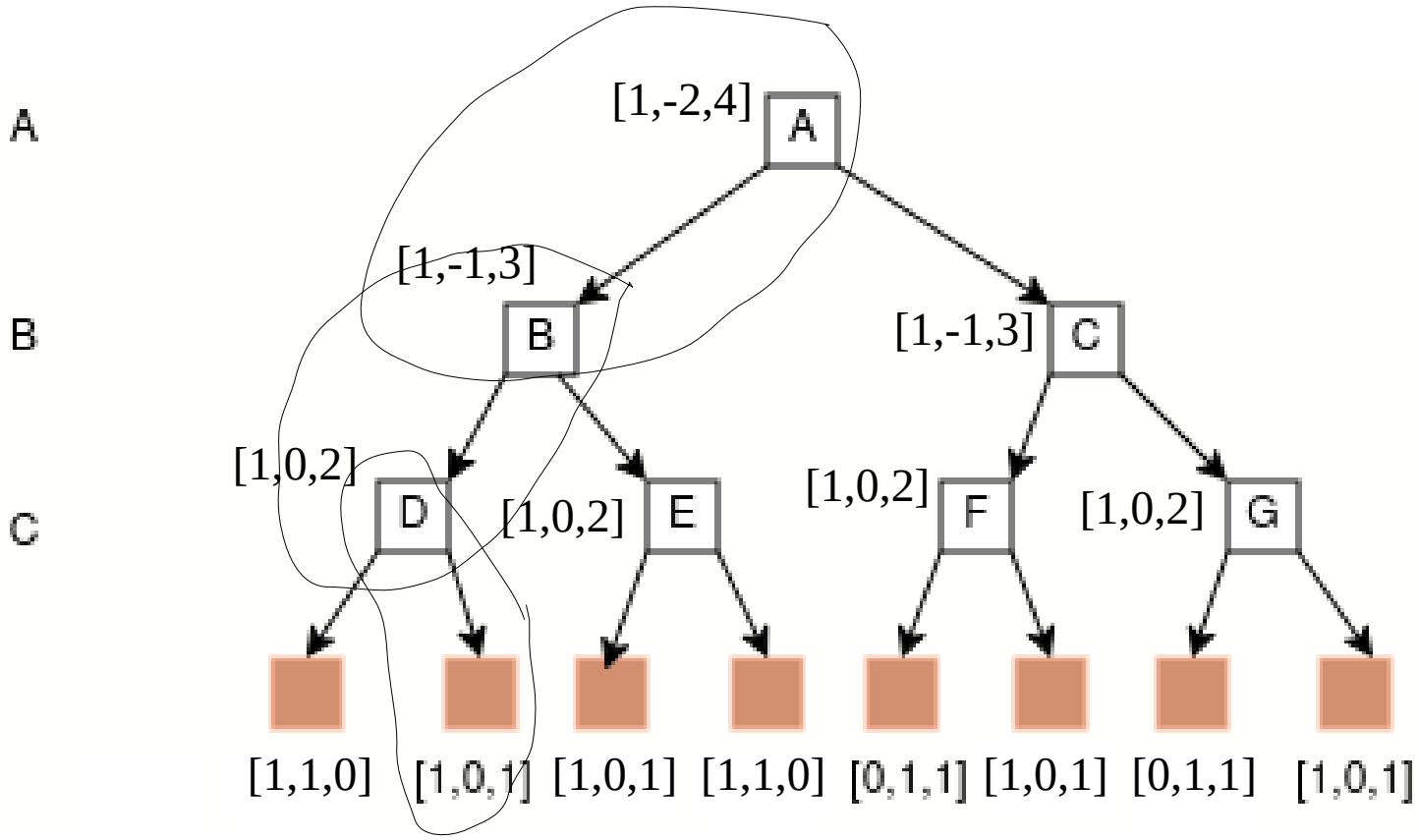


Note. K = 2

Multiplayer game via minimax: k-sum



Multiplayer game via minimax: k-sum



Limitation of minimax

- Time complexity is $O(b^m)$ where b = branching factor, m = max depth
  - This is significant challenge!
- We need to eliminate some of the nodes
  - No algorithm will eliminate the exponent m
  - We can prune a large number of the nodes

Reading

- Sections 5.1 - 5.2

